COMPANION TO YOUR PLACEMENT PDF

Placement Knowledge Base

145 pages, one topic each: the concept in depth, why it matters for your plan, key points, code, common mistakes, interview questions, practice problems and free resources. Tap a name below to jump to it; "Related" links at the end of each page jump between topics.

L0	L1	L2	L3	L4
Foundation	Core	Interview-ready	Advanced	Elite

Index

All 145 pages by category: concepts, 43 worked solutions and 5 model-answer sets. Tap to jump; ↑ Index on each page brings you back.

Method 4

- Active recall 4
- Spaced repetition (1-3-7-21) 5
- The 20–40 minute rule 6
- Patterns, not problems 7
- Interleaving 8
- Speaking while you code 9
- The error log 10
- Upsolving 11
- Deep-work blocks (50/10) 12
- Sleep, exercise and energy 13
- Weekly review 14
- Day modes and the never-zero rule 15
- Using AI as a tutor 16
- The five difficulty levels 17

DSA 18

- Time and space complexity 18
- C++ and the STL 19
- Hashing and frequency maps 20
- Two pointers 21
- Sliding window 22
- Prefix sums 23
- Binary search 24
- Binary search on the answer 25
- Linked lists 26
- Recursion and backtracking 27
- Stacks and queues 28
- Monotonic stack 29
- Heaps and priority queues 30
- Greedy algorithms 31
- Interval problems 32
- Bit manipulation 33
- Binary trees 34
- Binary search trees 35
- BFS and graph basics 36
- DFS, components and cycles 37
- Dijkstra and shortest paths 38
- Topological sort 39
- Disjoint set union (Union-Find) 40
- Dynamic programming 41
- Knapsack and subset DP 42
- String DP: LCS and edit distance 43
- Bitmask DP 44
- DP on trees 45
- Tries 46
- Segment trees and Fenwick trees 47
- String algorithms: KMP, Z, hashing 48
- Competitive programming and contests 49

Problems 50

- Two Sum 50
- Best Time to Buy and Sell Stock 51
- Product of Array Except Self 52
- Maximum Subarray (Kadane) 53
- Longest Consecutive Sequence 54
- Group Anagrams 55
- Top K Frequent Elements 56
- 3Sum 57
- Container With Most Water 58
- Trapping Rain Water 59
- Longest Substring Without Repeating Characters 60
- Minimum Window Substring 61
- Subarray Sum Equals K 62
- Search in Rotated Sorted Array 63
- Koko Eating Bananas 64
- Valid Parentheses 65
- Daily Temperatures 66
- Largest Rectangle in Histogram 67
- Reverse Linked List 68
- Linked List Cycle II 69
- Merge K Sorted Lists 70
- LRU Cache 71
- Binary Tree Level Order Traversal 72
- Lowest Common Ancestor of a Binary Tree 73
- Validate Binary Search Tree 74
- Binary Tree Maximum Path Sum 75
- Number of Islands 76
- Course Schedule 77
- Rotting Oranges 78
- Network Delay Time 79
- Number of Provinces 80
- House Robber 81
- Coin Change 82
- Longest Increasing Subsequence 83
- Longest Common Subsequence 84
- Edit Distance 85
- Partition Equal Subset Sum 86

Subsets 87
Merge Intervals 88
Kth Largest Element in an Array 89
Implement Trie (Prefix Tree) 90
Jump Game 91
Find Median from Data Stream 92

Core CS 93
Processes and threads 93
CPU scheduling 94
Deadlock 95
Synchronisation: mutex, semaphore, race conditions 96
Memory management and virtual memory 97
Normalisation and keys 98
Transactions, ACID and isolation 99
Indexing and B+ trees 100
SQL for interviews 101
OSI and TCP/IP models 102
TCP vs UDP and the handshake 103
HTTP, HTTPS and TLS 104
What happens when you type a URL 105
OOP: the four pillars 106
Virtual functions and the vtable 107
SOLID and design patterns 108

Q&A 109
Operating systems: model answers 109
DBMS and SQL: model answers 111
Computer networks: model answers 113
OOP and C++: model answers 115
Security and cloud: model answers 117

Design 119
Low-level design (LLD) 119
High-level system design (HLD) 120
Caching and CDNs 121
Load balancing, replication and sharding 122
CAP theorem and consistency 123

Security 124
OWASP Top 10 124
SQL injection 125
Cross-site scripting (XSS) and CSRF 126
Broken access control and IDOR 127
Server-side request forgery (SSRF) 128
Hashing, encryption and passwords 129
Threat modelling (STRIDE) 130
Binary exploitation basics 131
CTFs and security labs 132
Bug bounty and responsible disclosure 133

Cloud 134
AWS IAM and least privilege 134
ABAC: attribute-based access control 135
CloudTrail and CloudWatch alarms 136
TenantWall (your research project) 137
AWS core services and the SAA-C03 exam 138

AI 139
LLMs and RAG apps 139
Prompt injection and AI security 140

HFT 141
Probability for HFT interviews 141

Career 142
The interview loop (UMPIRE) 142
Online assessments (OAs) 143
Mock interviews 144
Behavioural interviews and STAR stories 145
The one-page resume 146
Projects that get shortlisted 147
Referrals and LinkedIn outreach 148
Cold emailing startups 149
The application pipeline 150
Preparing for a specific company 151
Offers, CTC and negotiation 152
Open source and GSoC 153

L0 · FOUNDATION METHOD

Active recall ↑ Index

> **WHY IT MATTERS FOR YOU**
> The single biggest learning multiplier. It's why the plan says 'solve before watching' and 'explain on a blank page'.

The concept

Retrieving information from memory strengthens it far more than re-reading it (the testing effect, Roediger & Karpicke 2006). Struggling to recall is the point; it feels slower but sticks.

In practice: close the video and solve, write a Core CS answer from memory before checking notes, and use flashcards that ask a question instead of showing a fact.

The Feynman technique: explain the concept in simple words on a blank page as if teaching a junior. Where you get vague is exactly what to re-learn.

Key points

- Notes in question form ('Why are threads cheaper?')
- Teach a friend or explain aloud
- Use the AI Feynman-checker prompt to find gaps

Common mistakes

× Highlighting and re-watching (feels productive, isn't)

Learn from

Roediger & Karpicke 2006 (PDF) (psychnet.wustl.edu) · Anki (apps.ankiweb.net)

Related: Spaced repetition (1-3-7-21) · The error log

L0 · FOUNDATION METHOD

Spaced repetition (1-3-7-21) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Without revision you forget most of a solved problem within weeks. This turns 550 problems into lasting skill.

The concept

Memory fades along a forgetting curve; each review at the right moment resets and flattens it. Spacing reviews out (1, 3, 7, 21 days) gives the most retention per minute.

For DSA: every problem you failed or needed a hint for goes on the list. On its due day, re-solve from a blank file in ≤ 10 minutes. Two clean passes in a row → retire it.

For Core CS facts: Anki does the scheduling automatically, 10 minutes a day.

Key points

- Revise before new problems each morning
- Keep revision to ~20% of DSA time
- Exam mode = revision only

Example

```
Google Sheets columns:
Date | Problem | Link | Pattern | Mistake | R1 | R3 | R7 | R21
R1: =A2+1   R3: =A2+3   R7: =A2+7   R21: =A2+21
Filter: any R column ⩽ TODAY() and not ticked
```

Common mistakes

× Re-reading old solutions instead of re-solving

Learn from

Anki manual (docs.ankiweb.net) · Gwern on spaced repetition (gwern.net)

Related: Active recall · The error log

L0 · FOUNDATION METHOD

The 20–40 minute rule ↑ Index

> **WHY IT MATTERS FOR YOU**
> Stops you wasting hours stuck, and stops you giving up too early. Both are common.

The concept

Easy/Medium: try for 20 minutes. Hard: 40 minutes. If stuck at the limit, read only the approach or hint (not code), then implement it yourself.

Stuck time is valuable up to a point: you are building the search for a pattern. Beyond it, you're just tired. The hint converts the struggle into learning.

Every problem you needed a hint for goes on the spaced-repetition list.

Key points

- Use the AI hint-only prompt for graded hints
- Read the editorial only until the key idea

Common mistakes

× Copying the full solution
× Spending 3 hours on one problem

Related: Spaced repetition (1-3-7-21) · The interview loop (UMPIRE) · Using AI as a tutor

L0 · FOUNDATION METHOD

Patterns, not problems ↑ Index

> **WHY IT MATTERS FOR YOU**
> 300 problems become about 25 reusable patterns, which is how strong candidates solve problems they've never seen.

The concept

After each solve, write one line: trigger → pattern. 'Sorted + pair sum → two pointers.' 'Minimum of the maximum → binary search on the answer.'

Before coding a new problem, scan your trigger list. If nothing fits, run the fallback checklist: can I sort it? hash it? binary search the answer? define states (DP)? model it as a graph?

Key points

- Keep the trigger list on one page
- Review it before every contest and interview

Common mistakes

× Memorising solutions instead of triggers

Learn from

NeetCode roadmap (neetcode.io)

Related: Two pointers · Sliding window · Binary search on the answer · Dynamic programming

L0 · FOUNDATION METHOD

Interleaving ↑ Index

> **WHY IT MATTERS FOR YOU**
> Interviews don't tell you the topic. Mixed practice trains you to identify the pattern, not just apply it.

The concept

Blocked practice (ten graph problems in a row) feels smooth because you already know the pattern. Interleaved practice (a graph, then a DP, then a string) forces you to choose, which is the real interview skill. Studies show interleaving improves later test performance even though it feels harder.

Use blocked practice when first learning a topic (Striver order), then interleave in revision days, contests and mocks.

Key points

- Sunday revision: random mix from the 1-3-7-21 list
- LeetCode 'pick one' or company-tagged lists are naturally interleaved

Common mistakes

× Interleaving before you know the basics of each topic

Related: Patterns, not problems · Spaced repetition (1-3-7-21)

L1 · CORE METHOD

Speaking while you code ↑ Index

> **WHY IT MATTERS FOR YOU**
> Interviewers score communication. Silence reads as 'stuck'. Tier-3 candidates practise this least.

The concept

Narrate: restate the problem, ask about constraints, say the brute force and its complexity, explain the better idea before coding, comment while coding ('this map stores value → index'), then dry-run aloud.

Practise one problem a day out loud, ideally recorded. Listen once a week for long silences and filler.

Key points

- Ask 'Shall I code this approach?' before coding
- Say complexity without being asked

Common mistakes

× Coding in silence for 10 minutes then explaining

Learn from

Pramp (free mock interviews) (pramp.com)

Related: The interview loop (UMPIRE) · Mock interviews

L0 · FOUNDATION METHOD

The error log ↑ Index

> **WHY IT MATTERS FOR YOU**
> Most wrong answers come from the same 10 mistakes. A log makes them visible so you stop repeating them.

The concept

Every wrong submission or failed interview question: one line with the problem, the mistake (off-by-one, int overflow, missed empty input, wrong pattern, misread question) and the fix.

Read the log before every contest, OA and interview. After a month, the top three mistake types become your personal pre-submit checklist.

Key points

- Tag mistakes by type to count them
- Pre-submit checklist: overflow? empty? n=1? duplicates? negative?

Common mistakes

× Logging without ever re-reading

Related: Spaced repetition (1-3-7-21) · Online assessments (OAs)

L1 · CORE METHOD

Upsolving ↑ Index

> **WHY IT MATTERS FOR YOU**
> The problem just past your level teaches the most; contests only show you where that is.

The concept

After a contest, take the first problem you couldn't solve. Try again for 30 minutes, then read the editorial idea, implement it yourself, and add it to the revision list.

One upsolved problem per contest beats two new contests without upsolving.

Key points

- Rating grows from upsolving, not from attempts

Common mistakes

× Skipping upsolving because the contest went badly

Learn from

Codeforces editorials (codeforces.com)

Related: Competitive programming and contests · The 20–40 minute rule

L0 · FOUNDATION METHOD

Deep-work blocks (50/10) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Hours at a desk don't measure progress; focused blocks do. Six blocks is a full-force day.

The concept

Work 50 minutes on one task with the phone in another room and notifications off; take a 10-minute break away from screens. Count blocks, not hours.

Put the hardest work (new DSA) in your sharpest hours, and revision, AWS and admin when tired. Decide each block's task the night before so you start instantly.

Key points

- Pomofocus timer
- Track blocks per day in your sheet
- 6 blocks = full force; 2 blocks = college day

Common mistakes

× Checking the phone 'just once' during a block

Learn from

Pomofocus (pomofocus.io) · Cal Newport, Deep Work (book) (calnewport.com)

Related: Day modes and the never-zero rule · Sleep, exercise and energy

L0 · FOUNDATION METHOD

Sleep, exercise and energy ↑ Index

> **WHY IT MATTERS FOR YOU**
> Memory consolidation happens during sleep. Cutting sleep to study more is a net loss.

The concept

Sleep 7–8 hours at a fixed time. Sleep after learning consolidates new memories; sleep deprivation hurts attention, problem-solving and recall the next day.

Exercise 20–30 minutes a day (walk, sport, gym) improves focus and mood. One rest half-day a week prevents burnout, which costs weeks.

Key points

- No all-nighters before contests or interviews
- Caffeine before 2 pm

Common mistakes

× Comparing yourself to LinkedIn posts at midnight

Learn from

Matthew Walker, Why We Sleep (book) (simonandschuster.com)

Related: Deep-work blocks (50/10) · Day modes and the never-zero rule

L0 · FOUNDATION METHOD

Weekly review ↑ Index

> **WHY IT MATTERS FOR YOU**
> Plans fail quietly. A 30-minute review each Sunday catches drift before it becomes a lost month.

The concept

Every Sunday: count problems, contests and blocks against the week's target; re-solve the week's failed problems; look at what slipped and why; set next week's three priorities.

Monthly (last Sunday, 1 hour): check the milestone checklist, take one timed mock on old topics, cut whatever isn't moving a number.

Key points

- Move unfinished items, never stack them on top
- Adjust quotas by ±1 problem, not ±5

Common mistakes

× Abandoning the plan after one bad week

Related: Day modes and the never-zero rule · Spaced repetition (1-3-7-21)

L0 · FOUNDATION METHOD

Day modes and the never-zero rule ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your weeks will differ: home, college, exams, bad days. A recipe for each removes the daily decision and keeps the streak.

The concept

Pick one mode each morning: Full force (6–8 blocks), College day (2 problems + AWS + flashcards), Exam mode (1 revision problem), Low energy (1 easy re-solve + read the error log), Sprint (mock + company set + applications).

Never zero: the streak is 'at least one problem touched'. A low-energy day that keeps the streak beats a perfect day followed by four days off.

Behind schedule? Cut the newest track (AI, then HFT), never DSA or revision, and never double tomorrow's quota.

Key points

- Plan tomorrow in 5 minutes tonight
- Exam mode 22 Mar – 7 Apr 2027

Common mistakes

× All-or-nothing thinking

Related: Deep-work blocks (50/10) · Weekly review

L0 · FOUNDATION METHOD

Using AI as a tutor ↑ Index

WHY IT MATTERS FOR YOU

Claude and Gemini can be a free tutor and interviewer, or a crutch that makes you feel productive while learning nothing. The rules decide which.

The concept

Use AI for: graded hints (one at a time), mock interviews, reviewing your accepted code, checking your Feynman explanations, generating flashcards, and resume critique.

Never paste a problem and ask for the solution before your 20/40-minute timebox. Interviewers spot borrowed understanding in two minutes of follow-ups.

Verify facts it gives you against docs; models can be confidently wrong, especially on exact numbers and APIs.

Key points

- Ask it to grade you 1–5 with one fix per area
- Use it to explain an editorial line you don't get

Example

```
Hint-only tutor prompt:
I'm solving [problem]. Don't give the solution or code. Give me one hint at a time,
the smallest nudge first. Wait for me to say 'next' before the next hint.
```

Common mistakes

× Copying AI code into LeetCode

× Trusting its complexity analysis without checking

Related: The 20–40 minute rule · Mock interviews

L0 · FOUNDATION METHOD

The five difficulty levels ↑ Index

> **WHY IT MATTERS FOR YOU**
> Tells you where to stop climbing on each topic, so you don't over-invest where your target jobs don't ask.

The concept

L0 Foundation: you can't do it yet; learn syntax, ideas and vocabulary. L1 Core: you can do it with notes (service-company level). L2 Interview-ready: cold, timed, out loud (product companies). L3 Advanced: you can handle the twist: Hards, contests, design trade-offs (Big Tech). L4 Elite: rare proof such as CF 1900+, CVEs, merged open-source work, HFT puzzles.

Your targets: DSA to L3, Core CS to L2, security to L3 (your edge), everything else to L1–L2 unless a specific job needs more.

Key points

- Climb in order
- Depth on one ladder beats breadth on five

Common mistakes

× Starting L4 topics before L2 is solid

Related: Dynamic programming · OWASP Top 10 · AWS core services and the SAA-C03 exam

L0 · FOUNDATION DSA

Time and space complexity ↑ Index

> **WHY IT MATTERS FOR YOU**
> Every interview answer ends with 'what's the complexity?'. It is also how you pick an approach before writing code: the input limit tells you which complexities will pass.

The concept

Big-O describes how running time or memory grows as the input size n grows, ignoring constants. A judge runs roughly 10^8 simple operations per second, so you can estimate before coding.

Read n from the constraints and work backwards: $n \le 10$ allows O(n!), $n \le 20$ allows $O(2^n)$, $n \le 500$ allows $O(n^3)$, $n \le 5000$ allows $O(n^2)$, $n \le 10^6$ needs O(n log n) or O(n), and $n \ge 10^9$ needs O(log n) or O(1) maths.

Space counts extra memory you allocate, including the recursion stack. A recursive DFS on a path of 10^5 nodes uses 10^5 stack frames, which can overflow.

Key points

- Nested loops over the same array → $O(n^2)$; halving each step → O(log n)
- Sorting is O(n log n); a hash map lookup is O(1) average, O(n) worst
- Amortised O(1): vector push_back doubles capacity occasionally
- Recursion: work per call × number of calls (draw the tree)
- Two pointers moving one way each → O(n) total, not $O(n^2)$

Common mistakes

× Forgetting the cost of string concatenation inside a loop
× Calling .size() or erase on the front of a vector inside a loop (erase is O(n))
× Saying O(1) for unordered_map without mentioning worst case when asked

Interview questions

☐ What is the complexity of your solution, time and space?
☐ Can you do better than $O(n^2)$?
☐ Why is building a heap O(n) and not O(n log n)?

Practise

Two Sum (leetcode.com) · Contains Duplicate (leetcode.com)

Learn from

Big-O cheat sheet (bigocheatsheet.com) · Striver A2Z sheet (takeuforward.org)

Related: C++ and the STL · Binary search

L0 · FOUNDATION DSA

C++ and the STL ↑ Index

> **WHY IT MATTERS FOR YOU**
> You chose C++ for DSA. In interviews and OAs, fluent STL saves 5–10 minutes per problem, and HFT firms test C++ more deeply than anyone.

The concept

Containers: vector (dynamic array), deque, list, set/map (red-black tree, ordered, O(log n)), unordered_set/unordered_map (hash table, O(1) average), priority_queue (binary heap, max by default), stack, queue.

Algorithms: sort, stable_sort, lower_bound/upper_bound (on sorted ranges), binary_search, reverse, accumulate, max_element, next_permutation, unique (with erase).

Idioms: auto and range-for, structured bindings (auto [a, b] = p), lambdas as comparators, emplace_back, fast I/O with ios::sync_with_stdio(false); cin.tie(nullptr).

Key points

- priority_queue<int, vector<int>, greater<int>> is a min-heap
- map[key] inserts a default if missing; use find or count to check
- lower_bound → first ≥ x; upper_bound → first > x
- Use long long when sums can pass $2 \cdot 10^9$
- Pass big containers by reference (const vector<int>&)

Code

```
vector<int> a = {5, 1, 4};
sort(a.begin(), a.end(), [](int x, int y) { return x > y; });
unordered_map<int,int> freq; for (int x : a) freq[x]++;
priority_queue<int, vector<int>, greater<int>> minh(a.begin(), a.end());
auto it = lower_bound(a.rbegin(), a.rend(), 4); // works on reversed order
```

Common mistakes

- × Iterator invalidation after erase/push_back
- × Comparator must be a strict weak ordering (use <, never <=)
- × Copying a vector into a function by value inside recursion

Interview questions

- ☐ Difference between map and unordered_map?
- ☐ How does vector grow?
- ☐ What is RAII?
- ☐ Why is unordered_map worst case O(n)?

Practise

Top K Frequent Elements (leetcode.com) · Group Anagrams (leetcode.com)

Learn from

takeuforward STL (takeuforward.org) · learncpp.com (learncpp.com) · cppreference (en.cppreference.com)

Related: Time and space complexity · OOP: the four pillars

L1 · CORE DSA

Hashing and frequency maps ↑ Index

> **WHY IT MATTERS FOR YOU**
> Turns many O(n²) 'have I seen this?' searches into O(n). It is the first optimisation interviewers expect.

The concept

A hash table stores keys in buckets chosen by a hash function. Average lookup, insert and delete are O(1); collisions make the worst case O(n).

Patterns: count frequencies, remember the index where a value was first seen, check complements (target − x), group by a canonical key (sorted string for anagrams).

Key points

- Two Sum: store value → index, check target − x before inserting x
- Anagram key: sorted string or 26-count array
- Use an array of 26/128 instead of a map when the alphabet is small

Common mistakes

× Inserting before checking (pairs an element with itself)

× Using unordered_map with pair keys (needs a custom hash)

Interview questions

☐ How does a hash map handle collisions?

☐ Why not always use a hash map?

Practise

Two Sum (leetcode.com) · Group Anagrams (leetcode.com) · Longest Consecutive Sequence (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Prefix sums · Sliding window

L1 · CORE DSA

Two pointers ↑ Index

> **WHY IT MATTERS FOR YOU**
> One of the five most frequent interview patterns. Turns pair searches in sorted data from O(n²) into O(n).

The concept

Keep two indices and move them based on a comparison. On a sorted array, if a[i] + a[j] is too small move i right; if too large move j left. Each pointer moves at most n times, so the total is O(n).

Variants: opposite ends (pair sum, container with most water), same direction fast/slow (remove duplicates, linked-list cycle), and fixing one element then running two pointers on the rest (3Sum, O(n²)).

Key points

- Needs sorted input or a monotone condition
- 3Sum: sort, fix i, two pointers on i+1..n−1, skip duplicates
- Partitioning (Dutch national flag) is three pointers

Code

```
sort(a.begin(), a.end());
int i = 0, j = n - 1;
while (i < j) {
    int s = a[i] + a[j];
    if (s == target) return {i, j};
    if (s < target) i++; else j--;
}
```

Common mistakes

- × Not skipping duplicates in 3Sum
- × Using it on unsorted data without sorting

Interview questions

- ☐ Why does moving the smaller side work in Container With Most Water?
- ☐ Prove two pointers doesn't miss a pair.

Practise

3Sum (leetcode.com) · Container With Most Water (leetcode.com) · Trapping Rain Water (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · NeetCode practice (neetcode.io)

Related: Sliding window · Binary search

L1 · CORE DSA

Sliding window ↑ Index

> **WHY IT MATTERS FOR YOU**
> Covers most 'longest/shortest substring or subarray such that…' questions, which appear in nearly every company's list.

The concept

Maintain a window [l, r]. Expand r each step; while the window breaks the rule, shrink l. Every index enters and leaves once, so it's O(n).

Fixed-size windows slide by adding a[r] and removing a[r−k]. Variable windows need the condition to be monotone: if a window is invalid, every bigger window containing it is invalid too.

'Exactly k' problems are often solved as atMost(k) − atMost(k−1).

Key points

- Track counts in a map or array for character windows
- Answer is updated after the shrink loop (longest) or inside it (shortest)
- Negative numbers break the monotone condition → use prefix sums instead

Code

```
int best = 0;
for (int l = 0, r = 0; r < n; r++) {
    add(a[r]);
    while (!valid()) remove(a[l++]);
    best = max(best, r - l + 1);
}
```

Common mistakes

- × Using it on sums with negative numbers
- × Forgetting to remove a[l] before l++

Interview questions

- ☐ Why is it O(n) when there's a nested while loop?
- ☐ How do you handle 'exactly k distinct'?

Practise

Longest Substring Without Repeating Characters (leetcode.com) · Minimum Window Substring (leetcode.com) · Longest Repeating Character Replacement (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Two pointers · Prefix sums · Hashing and frequency maps

L1 · CORE DSA

Prefix sums ↑ Index

> **WHY IT MATTERS FOR YOU**
> Range sums in O(1) and 'count subarrays with sum k' in O(n). Works with negative numbers where sliding window fails.

The concept

pre[i] = a[0] + … + a[i−1]. The sum of a[l..r] is pre[r+1] − pre[l].

To count subarrays summing to k: walk once, keep a map of how many times each prefix value has appeared; at each step add seen[pre − k]. Start with seen[0] = 1.

2D prefix sums give rectangle sums in O(1); difference arrays apply range updates in O(1) then rebuild.

Key points

- seen[0] = 1 handles subarrays starting at index 0
- Use long long for prefix arrays
- XOR prefix works the same way for XOR queries

Code

```
unordered_map<long long,int> seen{{0, 1}};
long long s = 0; int ways = 0;
for (int x : a) { s += x; ways += seen[s - k]; seen[s]++; }
```

Common mistakes

- × Off-by-one between pre[r+1] and pre[r]
- × Forgetting the initial seen[0] = 1

Interview questions

- ☐ Why doesn't a sliding window work for Subarray Sum Equals K?

Practise

Subarray Sum Equals K (leetcode.com) · Range Sum Query 2D Immutable (leetcode.com) · Continuous Subarray Sum (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Hashing and frequency maps · Sliding window · Segment trees and Fenwick trees

L1 · CORE DSA

Binary search ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked at every level, and the bug-prone template interviewers watch closely. Step 4 of Striver's sheet.

The concept

On a sorted range, compare with the middle and discard half. O(log n).

Think of it as finding the boundary of a predicate that is false…false then true…true. Search for the first true. This one template covers lower_bound, first/last occurrence, rotated arrays and peaks.

Use mid = lo + (hi − lo) / 2 to avoid overflow.

Key points

- Rotated sorted array: one half is always sorted; check which
- Peak element: compare a[mid] with a[mid+1]
- Search a 2D sorted matrix as a flattened 1D array

Code

```
int lo = 0, hi = n - 1, ans = n;
while (lo ≤ hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] ≥ x) { ans = mid; hi = mid - 1; }
    else lo = mid + 1;
}
```

Common mistakes

- × Infinite loop from lo = mid with lo < hi
- × Wrong boundary returned (first vs last)

Interview questions

- ☐ Find the first and last position of an element.
- ☐ Search in a rotated sorted array with duplicates.

Practise

Search In Rotated Sorted Array (leetcode.com) · Find First And Last Position Of Element In Sorted Array (leetcode.com) · Find Peak Element (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary search on the answer · Two pointers

L2 · INTERVIEW-READY DSA

Binary search on the answer ↑ Index

> **WHY IT MATTERS FOR YOU**
> A favourite Medium/Hard twist at Amazon, Google and in OAs. Recognising it is most of the work.

The concept

When the question asks for the minimum (or maximum) value X such that something is feasible, and feasibility is monotone in X, binary search over X and write an O(n) check(X).

Classic shapes: 'minimise the largest sum when splitting into k parts', 'minimum speed/capacity to finish in D days', 'maximise the minimum distance between cows'.

Key points

- Range of X: from the smallest possible answer to the largest
- check(X) is usually greedy
- Total O(n log range)

Code

```
long long lo = MIN_ANS, hi = MAX_ANS, ans = hi;
while (lo ⩽ hi) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) { ans = mid; hi = mid - 1; } else lo = mid + 1;
}
```

Common mistakes

× Wrong bounds (lo too high excludes the answer)

× Non-monotone check

Interview questions

☐ How do you know the answer space is monotone?

Practise

Koko Eating Bananas (leetcode.com) · Split Array Largest Sum (leetcode.com) · Capacity To Ship Packages Within D Days (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary search · Greedy algorithms

L1 · CORE DSA

Linked lists ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pointer manipulation questions are common in first rounds and test careful coding. Step 6 of the sheet.

The concept

Each node holds a value and a pointer to the next. Insertion/deletion at a known node is O(1); access by index is O(n).

Core techniques: a dummy head node to avoid edge cases, fast/slow pointers (middle, cycle detection by Floyd's algorithm, cycle start), and iterative reversal with prev/curr/next.

Key points

- Reverse: three pointers, return prev
- Cycle start: after meeting, move one pointer to head, step both by 1
- Merge two sorted lists with a dummy node
- LRU cache = hash map + doubly linked list

Code

```
ListNode* prev = nullptr; ListNode* cur = head;
while (cur) { ListNode* nx = cur->next; cur->next = prev; prev = cur; cur = nx; }
return prev;
```

Common mistakes

- × Dereferencing null on fast->next->next
- × Losing the rest of the list before re-linking

Interview questions

- ☐ Detect and remove a cycle.
- ☐ Reverse nodes in k-groups.
- ☐ Design an LRU cache.

Practise

Reverse Linked List (leetcode.com) · Linked List Cycle Ii (leetcode.com) · Lru Cache (leetcode.com) · Reverse Nodes In K Group (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Two pointers

L1 · CORE DSA

Recursion and backtracking ↑ Index

> **WHY IT MATTERS FOR YOU**
> The base for trees, graphs and DP. Subsets, permutations and N-Queens are standard interview questions.

The concept

Recursion: solve a problem using smaller copies of itself; define the base case first, then trust the call.

Backtracking explores a decision tree: choose, recurse, un-choose. It's exhaustive, so complexity is the size of the tree (2^n for subsets, n! for permutations).

Prune early: stop a branch as soon as it cannot lead to a valid answer.

Key points

- Pick / not-pick for subsets
- Swap-based or used[] array for permutations
- Skip duplicates by sorting and checking i > start && a[i] == a[i−1]

Code

```
void go(int i, vector<int>& cur) {
    if (i == n) { res.push_back(cur); return; }
    cur.push_back(a[i]); go(i + 1, cur); cur.pop_back(); // pick
    go(i + 1, cur);                                     // skip
}
```

Common mistakes

- × Forgetting to un-choose
- × Copying vectors in every call

Interview questions

- ☐ Generate all valid parentheses.
- ☐ Solve N-Queens.
- ☐ What's the complexity of your backtracking?

Practise

Subsets II (leetcode.com) · Permutations (leetcode.com) · Combination Sum (leetcode.com) · N Queens (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dynamic programming · Bitmask DP

L1 · CORE DSA

Stacks and queues ↑ Index

> **WHY IT MATTERS FOR YOU**
> Expression parsing, bracket matching and undo-style problems. Step 9 of the sheet, and the base for monotonic stacks.

The concept

A stack is last-in-first-out; a queue is first-in-first-out. Both give O(1) push and pop.

Use a stack when the most recent unfinished item matters (brackets, function calls, undo). Use a queue for processing in arrival order (BFS, scheduling).

Key points

- Valid parentheses with a stack of opening brackets
- Min-stack keeps a second stack of current minimums
- Queue using two stacks: amortised O(1)

Common mistakes

× Popping an empty stack

× Using std::stack when you need to iterate (use vector)

Interview questions

☐ Implement a queue with stacks.

☐ Design a min-stack.

Practise

Valid Parentheses (leetcode.com) · Min Stack (leetcode.com) · Evaluate Reverse Polish Notation (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Monotonic stack · BFS and graph basics

L2 · INTERVIEW-READY DSA

Monotonic stack ↑ Index

> **WHY IT MATTERS FOR YOU**
> Solves next-greater, stock span and largest-rectangle problems in O(n). A common Medium/Hard step-up.

The concept

Keep a stack whose values are always increasing (or decreasing). When a new element breaks the order, pop: each popped element has just found its next greater (or smaller) element.

Every element is pushed and popped once, so the total is O(n). Store indices, not values, so you can compute distances and widths.

Key points

- Largest rectangle in histogram: previous and next smaller
- Daily temperatures: next greater, answer = index distance
- Sum of subarray minimums: count contributions with previous/next smaller

Code

```
vector<int> nge(n, -1); stack<int> st;
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] < a[i]) { nge[st.top()] = a[i]; st.pop(); }
    st.push(i);
}
```

Common mistakes

× Strict vs non-strict comparison with duplicates (double counting)

Interview questions

☐ Largest rectangle in a histogram.
☐ Trapping rain water with a stack.

Practise

Daily Temperatures (leetcode.com) · Largest Rectangle In Histogram (leetcode.com) · Sum Of Subarray Minimums (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Stacks and queues

L2 · INTERVIEW-READY DSA

Heaps and priority queues ↑ Index

> **WHY IT MATTERS FOR YOU**
> Top-K, merging sorted lists, scheduling and Dijkstra all use a heap. Very common at Amazon.

The concept

A binary heap keeps the smallest (or largest) element on top. Push and pop are O(log n); peek is O(1); building from n items is O(n).

Top K largest: keep a min-heap of size k; pop when it grows past k. Streaming median: a max-heap for the lower half and a min-heap for the upper half, kept balanced.

Key points

- K-way merge: push the head of each list
- Use pair<int,int> to sort by priority then id
- Lazy deletion: skip stale entries when popped

Common mistakes

× Default priority_queue is a max-heap

× Sorting everything when you only need top k

Interview questions

☐ Find the median of a data stream.

☐ Why is heapify O(n)?

Practise

Kth Largest Element In An Array (leetcode.com) · Merge K Sorted Lists (leetcode.com) · Find Median From Data Stream (leetcode.com) · Task Scheduler (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dijkstra and shortest paths · Greedy algorithms

L2 · INTERVIEW-READY DSA

Greedy algorithms ↑ Index

> **WHY IT MATTERS FOR YOU**
> Short code, but you must justify it. Interviewers ask 'why does greedy work here?'.

The concept

Make the locally best choice at each step and never revisit it. It works only when an exchange argument holds: any optimal solution can be changed step by step into the greedy one without getting worse.

Most greedy solutions start by sorting: by end time (activity selection), by ratio (fractional knapsack), by deadline (job sequencing).

Key points

- Interval scheduling: sort by end, take if start ≥ last end
- Jump game: track the furthest reachable index
- If greedy fails on a small counterexample → it's DP

Common mistakes

× Assuming greedy works without testing a counterexample (0/1 knapsack)

Interview questions

☐ Prove your greedy choice is safe.

☐ When would greedy fail here?

Practise

Jump Game Ii (leetcode.com) · Non Overlapping Intervals (leetcode.com) · Gas Station (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Interval problems · Dynamic programming

L2 · INTERVIEW-READY DSA

Interval problems ↑ Index

> **WHY IT MATTERS FOR YOU**
> Merge intervals and meeting rooms are top-asked at Google, Microsoft and Amazon.

The concept

Sort by start (to merge) or by end (to choose the most non-overlapping). Then sweep once.

For 'how many overlap at once' (meeting rooms), sweep over events: +1 at start, −1 at end, and track the maximum; or use a min-heap of end times.

Key points

- Merge: if start ≤ last end, extend; else push new
- Insert interval: three phases (before, overlapping, after)

Common mistakes

× Touching intervals: decide whether [1,2] and [2,3] overlap

Interview questions

☐ Minimum meeting rooms needed.
☐ Merge overlapping intervals.

Practise

Merge Intervals (leetcode.com) · Insert Interval (leetcode.com) · Non Overlapping Intervals (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Greedy algorithms · Heaps and priority queues

L1 · CORE DSA

Bit manipulation ↑ Index

> **WHY IT MATTERS FOR YOU**
> Quick wins in OAs (single number, power of two), and the base for bitmask DP.

The concept

x & 1 tests odd; x & (x−1) clears the lowest set bit; x & −x isolates it; a ^ a = 0, so XOR of all elements leaves the one that appears once.

A mask of n bits represents a subset of n items: loop mask from 0 to $2^n - 1$ to enumerate all subsets.

Key points

- __builtin_popcount(x) counts set bits
- Check bit i: (x >> i) & 1; set: x | (1 << i)
- Use 1LL << i for i ≥ 31

Common mistakes

× Operator precedence: write (x & 1) == 0 with brackets

Interview questions

☐ Find the two numbers that appear once.

☐ Count bits from 0 to n.

Practise

Single Number (leetcode.com) · Counting Bits (leetcode.com) · Subsets (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · cp-algorithms: bit manipulation (cp-algorithms.com)

Related: Bitmask DP · Tries

L2 · INTERVIEW-READY DSA

Binary trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> The largest single topic in interviews. Step 13–14 of the sheet.

The concept

Traversals: preorder, inorder, postorder (DFS) and level order (BFS with a queue). Know both recursive and iterative versions.

Most tree problems are 'return something from each subtree and combine': height, diameter, balanced check, max path sum. Write a function that returns the subtree answer and updates a global best.

Lowest common ancestor: if both sides return non-null, the current node is the LCA.

Key points

- Diameter = max over nodes of leftHeight + rightHeight
- Serialize with preorder + null markers
- Views (left/right/top/bottom) use BFS with column index

Code

```
int best = INT_MIN;
int gain(TreeNode* r) {
    if (!r) return 0;
    int L = max(0, gain(r->left)), R = max(0, gain(r->right));
    best = max(best, r->val + L + R);
    return r->val + max(L, R);
}
```

Common mistakes

- × Confusing 'path through node' with 'path returned to parent'
- × Recursion depth on skewed trees

Interview questions

- ☐ Maximum path sum.
- ☐ Serialize and deserialize a tree.
- ☐ Lowest common ancestor.

Practise

Binary Tree Maximum Path Sum (leetcode.com) · Lowest Common Ancestor Of A Binary Tree (leetcode.com) · Serialize And Deserialize Binary Tree (leetcode.com) · Diameter Of Binary Tree (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary search trees · DP on trees · BFS and graph basics

L2 · INTERVIEW-READY DSA

Binary search trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> Common follow-up to tree questions; inorder-sorted property shows up everywhere.

The concept

Left subtree < node < right subtree. Inorder traversal gives sorted order. Search, insert and delete are O(h): O(log n) if balanced, O(n) if skewed.

Validate with a (low, high) range passed down, not just comparing with children.

Key points

- Kth smallest = kth node in inorder
- LCA in BST: walk down using values
- std::set/map are balanced BSTs (red-black)

Common mistakes

× Validating only parent-child pairs

Interview questions

☐ Validate a BST.

☐ Delete a node from a BST.

Practise

Validate Binary Search Tree (leetcode.com) · Kth Smallest Element In A Bst (leetcode.com) · Delete Node In A Bst (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Binary trees · Binary search

L1 · CORE DSA

BFS and graph basics ↑ Index

> **WHY IT MATTERS FOR YOU**
> Graphs are in almost every product-company loop. BFS gives shortest paths when every edge costs the same.

The concept

Store a graph as an adjacency list: vector<vector<int>> adj(n). Grids are implicit graphs with 4 neighbours.

BFS uses a queue and visits nodes in order of distance from the source, so the first time you reach a node is its shortest path. O(V + E).

Multi-source BFS starts with all sources in the queue (rotting oranges, nearest 0). 0-1 BFS uses a deque for edges of weight 0 or 1.

Key points

- Mark visited when pushing, not when popping
- Grid: dr/dc arrays for directions
- Bipartite check = BFS 2-colouring

Code

```
vector<int> dist(n, -1); queue<int> q; dist[s] = 0; q.push(s);
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
}
```

Common mistakes

× Marking visited on pop → duplicates in queue

Interview questions

☐ Rotting oranges.
☐ Word ladder.
☐ Is the graph bipartite?

Practise

Rotting Oranges (leetcode.com) · Word Ladder (leetcode.com) · Is Graph Bipartite (leetcode.com) · Number Of Islands (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · CSES problem set (cses.fi)

Related: DFS, components and cycles · Dijkstra and shortest paths · Topological sort

L1 · CORE DSA

DFS, components and cycles ↑ Index

> **WHY IT MATTERS FOR YOU**
> Connected components, cycle detection and flood fill are standard graph questions.

The concept

DFS goes deep along one path before backtracking. Recursive or with an explicit stack. O(V + E).

Undirected cycle: you reach a visited node that isn't your parent. Directed cycle: you reach a node still on the current recursion path (keep an inStack array or 3 colours).

Count components by starting a DFS from every unvisited node.

Key points

- Flood fill on grids is DFS
- Directed cycle check = topo sort fails
- Bridges/articulation points use discovery and low times (L3)

Common mistakes

× Stack overflow on 10^6-node grids: use iterative DFS or BFS

Interview questions

☐ Detect a cycle in a directed graph.
☐ Number of islands.
☐ Clone a graph.

Practise

Number Of Islands (leetcode.com) · Clone Graph (leetcode.com) · Course Schedule (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · CSES problem set (cses.fi)

Related: BFS and graph basics · Topological sort · Disjoint set union (Union-Find)

L2 · INTERVIEW-READY DSA

Dijkstra and shortest paths ↑ Index

> **WHY IT MATTERS FOR YOU**
> The standard weighted shortest path. Asked at Google, Uber, Flipkart.

The concept

Dijkstra: a min-heap of (distance, node). Pop the closest, relax its edges. Works only with non-negative weights. O((V + E) log V).

Bellman-Ford relaxes all edges V−1 times, handles negative edges and detects negative cycles, O(VE). Floyd-Warshall gives all pairs in $O(V^3)$ for small V.

Key points

- Skip stale heap entries: if (d > dist[u]) continue
- Cheapest flights within K stops: BFS by level or Bellman-Ford with K rounds
- Path reconstruction with a parent array

Code

```
vector<long long> d(n, INF); d[s] = 0;
priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq; pq.push({0, s});
while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du > d[u]) continue;
    for (auto [v, w] : adj[u]) if (d[u] + w < d[v]) { d[v] = d[u] + w; pq.push({d[v], v}); }
}
```

Common mistakes

- × Using Dijkstra with negative edges
- × int overflow on distances

Interview questions

- ☐ Why does Dijkstra fail with negative weights?
- ☐ Network delay time.

Practise

Network Delay Time (leetcode.com) · Path With Minimum Effort (leetcode.com) · Cheapest Flights Within K Stops (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · cp-algorithms: Dijkstra (cp-algorithms.com)

Related: BFS and graph basics · Heaps and priority queues

L2 · INTERVIEW-READY DSA

Topological sort ↑ Index

> **WHY IT MATTERS FOR YOU**
> Course schedule and build-order questions; also how you detect cycles in directed graphs.

The concept

An order of a DAG's nodes where every edge goes forward. Kahn's algorithm: push all nodes with in-degree 0, pop, decrement neighbours, push any that reach 0. If you output fewer than V nodes, there's a cycle.

DFS version: add a node to the list after visiting all its children, then reverse.

Key points

- Alien dictionary: build edges from adjacent words
- Longest path in a DAG = DP in topo order

Common mistakes

× Not handling disconnected nodes

Interview questions

☐ Course schedule II.
☐ Alien dictionary.

Practise

Course Schedule Ii (leetcode.com) · Course Schedule (leetcode.com) · Parallel Courses (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: DFS, components and cycles · BFS and graph basics

L2 · INTERVIEW-READY DSA

Disjoint set union (Union-Find) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Fast connectivity for dynamic merging, Kruskal's MST and 'accounts merge'.

The concept

Each set has a root. find(x) follows parents to the root; union(a, b) links roots. With path compression and union by size, each operation is almost O(1) (inverse Ackermann).

Kruskal's minimum spanning tree: sort edges by weight, add an edge if its ends are in different sets.

Key points

- Count components = number of successful unions subtracted from n
- Redundant connection: the edge whose ends are already joined

Code

```
vector<int> p(n), sz(n, 1); iota(p.begin(), p.end(), 0);
function<int(int)> find = [&](int x) { return p[x] == x ? x : p[x] = find(p[x]); };
auto unite = [&](int a, int b) { a = find(a); b = find(b); if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b); p[b] = a; sz[a] += sz[b]; return true; };
```

Common mistakes

× Forgetting to call find before comparing

Interview questions

☐ Number of provinces.
☐ Accounts merge.
☐ Minimum spanning tree.

Practise

Number Of Provinces (leetcode.com) · Redundant Connection (leetcode.com) · Accounts Merge (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · cp-algorithms: DSU (cp-algorithms.com)

Related: DFS, components and cycles

L2 · INTERVIEW-READY DSA

Dynamic programming

↑ Index

> **WHY IT MATTERS FOR YOU**
> The topic that separates L2 from L3. Striver step 16 is the longest; product companies ask at least one DP.

The concept

DP = recursion + remembering answers to overlapping subproblems. Recipe: (1) define the state in words, e.g. dp[i] = best answer for the first i items; (2) write the transition from smaller states; (3) base cases; (4) the order to fill; (5) where the answer is.

Go memoised recursion → bottom-up table → space-optimised (keep only the last row). Interviewers love seeing all three.

Families: 1D (climbing stairs, house robber), grid paths, 0/1 and unbounded knapsack, subsequences (LIS, LCS, edit distance), partitions (palindrome partitioning, matrix chain), stocks (state machine), DP on trees, bitmask DP.

Key points

- Complexity = number of states × work per transition
- If greedy fails on a counterexample, think DP
- Print the solution by walking back through choices

Code

```
// 0/1 knapsack, space-optimised
vector<int> dp(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int w = W; w >= wt[i]; w--)
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
```

Common mistakes

- × Iterating capacity upwards in 0/1 knapsack (turns it unbounded)
- × Wrong base case for empty prefix

Interview questions

- ☐ Explain your state and transition.
- ☐ Can you reduce the space?
- ☐ Edit distance.
- ☐ Longest increasing subsequence in O(n log n).

Practise

House Robber (leetcode.com) · Coin Change (leetcode.com) · Longest Increasing Subsequence (leetcode.com) · Edit Distance (leetcode.com) · Partition Equal Subset Sum (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org) · AtCoder Educational DP contest (atcoder.jp)

Related: Knapsack and subset DP · String DP: LCS and edit distance · Bitmask DP · DP on trees · Recursion and backtracking

L2 · INTERVIEW-READY DSA

Knapsack and subset DP ↑ Index

> **WHY IT MATTERS FOR YOU**
> Subset-sum, partition and coin-change are all knapsack in disguise.

The concept

0/1 knapsack: each item once; loop capacity downwards. Unbounded (coin change): items reusable; loop capacity upwards.

Subset sum to target T: boolean dp[t]. Partition equal subset: subset sum to total/2. Count of subsets with difference d: subset sum to (total + d)/2.

Key points

- Coin change (min coins): dp[x] = min(dp[x − c] + 1)
- Coin change II (count ways): loop coins outside, amounts inside

Common mistakes

× Order of loops changes combinations vs permutations

Interview questions

☐ Target sum.

☐ Partition equal subset sum.

Practise

Coin Change Ii (leetcode.com) · Target Sum (leetcode.com) · Partition Equal Subset Sum (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dynamic programming

L2 · INTERVIEW-READY DSA

String DP: LCS and edit distance ↑ Index

> **WHY IT MATTERS FOR YOU**
> Two-string comparison problems are a fixed pattern; once learned, a whole family falls.

The concept

dp[i][j] = answer for prefixes a[0..i) and b[0..j). If a[i−1] == b[j−1], extend the diagonal; otherwise take the best of dropping one character.

Edit distance: min of insert dp[i][j−1], delete dp[i−1][j], replace dp[i−1][j−1], each +1. Shortest common supersequence and longest palindromic subsequence (LCS of s and reverse(s)) follow.

Key points

- Space-optimise to two rows
- Distinct subsequences counts ways

Common mistakes

× Off-by-one between string index and dp index

Interview questions

☐ Edit distance.

☐ Longest common subsequence and print it.

Practise

Longest Common Subsequence (leetcode.com) · Edit Distance (leetcode.com) · Distinct Subsequences (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Dynamic programming · String algorithms: KMP, Z, hashing

L3 · ADVANCED DSA

Bitmask DP ↑ Index

> **WHY IT MATTERS FOR YOU**
> L3/L4 topic for hard rounds, OAs at top firms and HFT. Shows up when n ≤ 20.

The concept

The state includes a mask of which items are used: dp[mask] or dp[mask][last]. 2^n masks × n transitions → $O(2^n \cdot n)$ or $O(2^n \cdot n^2)$.

Travelling salesman: dp[mask][i] = shortest path visiting mask and ending at i.

Key points

- Iterate submasks: for (s = m; s; s = (s − 1) & m)
- Memory: 2^{20} × 20 ints ≈ 80 MB, watch limits

Common mistakes

× Using it when n is 40 (meet-in-the-middle instead)

Interview questions

☐ Shortest path visiting all nodes.

Practise

Shortest Path Visiting All Nodes (leetcode.com) · Partition To K Equal Sum Subsets (leetcode.com) · CSES: Elevator Rides (cses.fi)

Learn from

cp-algorithms: submask enumeration (cp-algorithms.com) · CSES problem set (cses.fi)

Related: Dynamic programming · Bit manipulation

L3 · ADVANCED DSA

DP on trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> Hard-round favourite: house robber III, max path sum, rerooting.

The concept

Compute dp for each node from its children in a post-order DFS. Often each node returns a small tuple (take, skip).

Rerooting: compute answers for one root, then move the root to each child in O(1) to get answers for all roots in O(n).

Key points

- House Robber III returns {rob, notRob}
- Sum of distances in tree is the classic rerooting problem

Common mistakes

× Recomputing subtrees ($O(n^2)$)

Interview questions

☐ House robber on a tree.

Practise

House Robber Iii (leetcode.com) · Sum Of Distances In Tree (leetcode.com) · CSES: Tree Distances I (cses.fi)

Learn from

USACO Guide: DP on trees (usaco.guide) · CSES problem set (cses.fi)

Related: Binary trees · Dynamic programming

L2 · INTERVIEW-READY DSA

Tries ↑ Index

> **WHY IT MATTERS FOR YOU**
> Autocomplete, word search and max-XOR problems. Last step of Striver's sheet.

The concept

A tree where each edge is a character; the path from the root spells a prefix. Insert and search are O(word length).

A bit trie stores numbers bit by bit from the top; to maximise XOR with x, greedily go to the opposite bit at each level.

Key points

- Node: array of 26 children + end flag (+ count for prefix counts)
- Word Search II: trie + DFS on the grid

Code

```
struct Node { Node* ch[26] = {}; bool end = false; };
void insert(Node* r, const string& w) {
    for (char c : w) { if (!r->ch[c - 'a']) r->ch[c - 'a'] = new Node(); r = r->ch[c - 'a']; }
    r->end = true;
}
```

Common mistakes

× Memory: 26 pointers per node; use array pools for big inputs

Interview questions

☐ Implement a trie.

☐ Maximum XOR of two numbers.

Practise

Implement Trie Prefix Tree (leetcode.com) · Word Search Ii (leetcode.com) · Maximum Xor Of Two Numbers In An Array (leetcode.com)

Learn from

Striver A2Z sheet (takeuforward.org)

Related: Bit manipulation · String algorithms: KMP, Z, hashing

L3 · ADVANCED DSA

Segment trees and Fenwick trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> L3: range queries with updates. Needed for CP, CSES and harder OAs (Google, D. E. Shaw).

The concept

A segment tree stores an aggregate (sum, min, max) for each segment; query and point update are O(log n). With lazy propagation it supports range updates too.

A Fenwick tree (BIT) is shorter code for prefix sums with point updates, O(log n) each, using i & −i.

Key points

- Size the segment tree array 4n
- Count inversions with a BIT over compressed values

Code

```
// Fenwick, 1-indexed
vector<long long> bit(n + 1);
void add(int i, long long v) { for (; i ⩽ n; i += i & -i) bit[i] += v; }
long long sum(int i) { long long s = 0; for (; i > 0; i -= i & -i) s += bit[i]; return s; }
```

Common mistakes

× Forgetting to push lazy values down before recursing

Interview questions

☐ Range sum query, mutable.

☐ Count of smaller numbers after self.

Practise

Range Sum Query Mutable (leetcode.com) · Count Of Smaller Numbers After Self (leetcode.com) · CSES: Range Queries (cses.fi)

Learn from

cp-algorithms: segment tree (cp-algorithms.com) · cp-algorithms: Fenwick tree (cp-algorithms.com)

Related: Prefix sums

L3 · ADVANCED DSA

String algorithms: KMP, Z, hashing ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern matching in O(n + m). Appears in hard rounds and CP.

The concept

KMP builds a prefix function: pi[i] = length of the longest proper prefix of s[0..i] that is also a suffix. Matching never moves backwards in the text.

The Z-function z[i] = length of the longest substring starting at i that matches a prefix. Search pattern + '$' + text.

Rolling hash (Rabin-Karp) compares substrings in O(1) after O(n) preprocessing; use two moduli to reduce collisions.

Key points

- Shortest palindrome: KMP on s + '#' + reverse(s)
- Repeated substring pattern via prefix function

Common mistakes

× Single-modulus hashing can be broken by anti-hash tests

Interview questions

☐ Find the first occurrence of a pattern in O(n + m).

Practise

Find The Index Of The First Occurrence In A String (leetcode.com) · Shortest Palindrome (leetcode.com) · Repeated Substring Pattern (leetcode.com)

Learn from

cp-algorithms: prefix function (cp-algorithms.com) · cp-algorithms: Z-function (cp-algorithms.com) · cp-algorithms: string hashing (cp-algorithms.com)

Related: Tries · String DP: LCS and edit distance

L3 · ADVANCED DSA

Competitive programming and contests ↑ Index

> **WHY IT MATTERS FOR YOU**
> Ratings are a filter recruiters understand, contests train speed under pressure (exactly what OAs test), and HFT firms hire heavily from strong CP profiles.

The concept

Do one LeetCode weekly contest and one Codeforces Div 2/3 round a week. After each, upsolve the first problem you couldn't solve.

The CP31 sheet gives 31 problems per rating band (800 → 1900): climb in order. CSES gives 300 classic problems by topic.

Targets from your plan: LC 1800+ or CF 1400+ by mid-2027 for Big Tech; CF 1600–1900+ for HFT.

Key points

- Read all problems first; solve in order of confidence
- Stress-test with a brute force on random inputs
- Keep a contest log: problem, idea, mistake

Common mistakes

× Grinding new contests without upsolving
× Chasing rating over interview skills before L2 is done

Interview questions

☐ (Interviews rarely ask about CP; your rating speaks on the resume)

Practise

Codeforces (codeforces.com) · LeetCode contests (leetcode.com) · CSES problem set (cses.fi) · AtCoder (atcoder.jp)

Learn from

USACO Guide (usaco.guide) · cp-algorithms (cp-algorithms.com) · clist.by contest calendar (clist.by)

Related: Online assessments (OAs) · Time and space complexity

Two Sum ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Hashing. The most famous first-round question; asked everywhere as a warm-up.

The idea

Brute force checks every pair: $O(n^2)$. Instead, walk once and remember each value's index in a hash map. For each x, the partner you need is target − x; if it's already in the map, you're done.

Check the map before inserting x, so an element is never paired with itself.

Steps

- For i in 0..n−1: if target − a[i] is in the map, return [map[target − a[i]], i]
- Otherwise store a[i] → i
- Complexity: O(n) time, O(n) space

Example

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> pos;                // value → index
        for (int i = 0; i < (int)nums.size(); i++) {
            auto it = pos.find(target - nums[i]);
            if (it ≠ pos.end()) return {it→second, i};
            pos[nums[i]] = i;
        }
        return {};
    }
};
```

Common mistakes

- × Inserting before checking (pairs x with itself)
- × Returning values instead of indices

Follow-up questions

- ☐ What if the array is sorted? (two pointers, O(1) space)
- ☐ What if there can be many answers? Return all pairs without duplicates.

Practise

Solve it on LeetCode: Two Sum (leetcode.com)

Related: Hashing and frequency maps

L1 · CORE PROBLEMS

Best Time to Buy and Sell Stock ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Greedy. Amazon and Microsoft favourite; leads into the stock DP family.

The idea

The best sale on day i uses the cheapest price before day i. Track the running minimum and the best difference.

Steps

- minPrice = a[0]; best = 0
- For each price: best = max(best, price − minPrice); minPrice = min(minPrice, price)
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int lo = INT_MAX, best = 0;
        for (int p : prices) { best = max(best, p - lo); lo = min(lo, p); }
        return best;
    }
};
```

Common mistakes

× p − INT_MAX overflow if the order of the two lines is swapped carelessly (here best is updated first with lo = INT_MAX, which gives a negative number, safe)

Follow-up questions

☐ Unlimited transactions? (sum every positive difference)

☐ At most k transactions or with cooldown? (state-machine DP)

Practise

Solve it on LeetCode: Best Time to Buy and Sell Stock (leetcode.com)

Related: Greedy algorithms

Product of Array Except Self ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Prefix sum. Asked at Amazon, Meta, Microsoft; tests prefix thinking without division.

The idea

answer[i] = (product of everything left of i) × (product of everything right of i). Fill left products in one pass, then multiply by a running right product in a second pass.

Steps

- res[i] = product of a[0..i−1]
- Walk from the right with r = 1: res[i] *= r; r *= a[i]
- Complexity: O(n) time, O(1) extra space (the output doesn't count)

Example

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> res(n, 1);
        for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
        int r = 1;
        for (int i = n - 1; i ⩾ 0; i--) { res[i] *= r; r *= nums[i]; }
        return res;
    }
};
```

Common mistakes

× Using division (breaks with zeros, and it's forbidden)

Follow-up questions

☐ Why is the output array not counted as extra space?

Practise

Solve it on LeetCode: Product of Array Except Self (leetcode.com)

Related: Prefix sums

L2 · INTERVIEW-READY PROBLEMS

Maximum Subarray (Kadane) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Kadane. Classic; tests whether you see the 1-D DP hidden in it.

The idea

At each index, the best subarray ending here either extends the best one ending at i−1 or starts fresh at i: cur = max(a[i], cur + a[i]). The answer is the best cur seen.

Steps

- cur = best = a[0]
- For i ≥ 1: cur = max(a[i], cur + a[i]); best = max(best, cur)
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int cur = nums[0], best = nums[0];
        for (int i = 1; i < (int)nums.size(); i++) { cur = max(nums[i], cur + nums[i]); best = max(best,
cur); }
        return best;
    }
};
```

Common mistakes

× Starting best at 0 (wrong when all numbers are negative)

Follow-up questions

☐ Return the start and end indices too.

☐ Maximum product subarray? (track both max and min)

Practise

Solve it on LeetCode: Maximum Subarray (Kadane) (leetcode.com)

Related: Dynamic programming

L2 · INTERVIEW-READY PROBLEMS

Longest Consecutive Sequence ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Hashing. Google and Amazon; the O(n) requirement is the whole point.

The idea

Put all numbers in a hash set. Only start counting from x when x − 1 is not in the set (x is the start of a run), then walk x+1, x+2... Each number is visited at most twice, so the total is O(n).

Steps

- set s(all)
- For x in s: if x−1 not in s, count the run from x
- Complexity: O(n) average time, O(n) space

Example

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int best = 0;
        for (int x : s) {
            if (s.count(x - 1)) continue;          // not the start of a run
            int len = 1;
            while (s.count(x + len)) len++;
            best = max(best, len);
        }
        return best;
    }
};
```

Common mistakes

× Starting a walk from every number (O(n^2) on a long run)
× Sorting (O(n log n), not what's asked)

Follow-up questions

☐ Why is it O(n) even with the inner while loop?

Practise

Solve it on LeetCode: Longest Consecutive Sequence (leetcode.com)

Related: Hashing and frequency maps

Group Anagrams ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Hashing. Common at Amazon, Uber and service-company OAs.

The idea

Anagrams share a canonical key. Use the sorted word (O(k log k)) or a 26-letter count string (O(k)) as the hash-map key, and append each word to its group.

Steps

- key = sorted(word)
- groups[key].push_back(word)
- Complexity: O(n·k log k) time, O(n·k) space

Example

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> g;
        for (auto& s : strs) { string k = s; sort(k.begin(), k.end()); g[k].push_back(s); }
        vector<vector<string>> res;
        for (auto& [k, v] : g) res.push_back(v);
        return res;
    }
};
```

Common mistakes

× Using a count array as the key without separators ("1","11" collisions)

Follow-up questions

☐ Make the key O(k) instead of O(k log k).

Practise

Solve it on LeetCode: Group Anagrams (leetcode.com)

Related: Hashing and frequency maps

Top K Frequent Elements ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Heap. Amazon's most-asked heap problem.

The idea

Count frequencies with a hash map, then keep a min-heap of size k keyed by frequency; anything smaller than the heap's top can't be in the answer. Bucket sort by frequency gives O(n).

Steps

- Count
- Push (freq, value) into a min-heap; pop when size > k
- Heap contents are the answer
- Complexity: O(n log k) time, O(n) space

Example

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int, int> f;
        for (int x : nums) f[x]++;
        priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;   // (freq, value), min on top
        for (auto& [v, c] : f) { pq.push({c, v}); if ((int)pq.size() > k) pq.pop(); }
        vector<int> res;
        while (!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
        return res;
    }
};
```

Common mistakes

× Using a max-heap of all n elements (O(n log n))

Follow-up questions

☐ Do it in O(n) with buckets.

☐ What if the data is a stream?

Practise

Solve it on LeetCode: Top K Frequent Elements (leetcode.com)

Related: Heaps and priority queues

3Sum ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Two pointers. Top-10 at Meta, Amazon, Microsoft; tests duplicate handling.

The idea

Sort. Fix a[i] and find pairs in a[i+1..n−1] summing to −a[i] with two pointers. Skip equal values of a[i], and skip equal values after finding a triple, so no duplicate triples appear.

Steps

- Sort
- For each i (skip a[i] == a[i−1]): l = i+1, r = n−1
- Move l or r by the sum's sign; on a hit, record and skip duplicates on both sides
- Complexity: O(n^2) time, O(1) extra space (besides sorting)

Example

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& a) {
        sort(a.begin(), a.end());
        vector<vector<int>> res; int n = a.size();
        for (int i = 0; i < n - 2; i++) {
            if (i && a[i] == a[i - 1]) continue;
            int l = i + 1, r = n - 1;
            while (l < r) {
                int s = a[i] + a[l] + a[r];
                if (s < 0) l++;
                else if (s > 0) r--;
                else {
                    res.push_back({a[i], a[l], a[r]});
                    while (l < r && a[l] == a[l + 1]) l++;
                    while (l < r && a[r] == a[r - 1]) r--;
                    l++; r--;
                }
            }
        }
        return res;
    }
};
```

Common mistakes

× Not skipping duplicates
× Using a set of triples to dedupe (slow, and interviewers notice)

Follow-up questions

☐ 4Sum?
☐ 3Sum closest?

Practise

Solve it on LeetCode: 3Sum (leetcode.com)

Related: Two pointers

Container With Most Water ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Two pointers. Asked at Google and Amazon; the proof is the interview.

The idea

Start with the widest container. The height is limited by the shorter line, so moving the taller line inward can never help: the width shrinks and the height can't exceed the shorter line. Always move the shorter one.

Steps

- l = 0, r = n−1
- area = (r − l) × min(h[l], h[r])
- Move the shorter side inward
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    int maxArea(vector<int>& h) {
        int l = 0, r = h.size() - 1, best = 0;
        while (l < r) {
            best = max(best, (r - l) * min(h[l], h[r]));
            if (h[l] < h[r]) l++; else r--;
        }
        return best;
    }
};
```

Common mistakes

× Moving the taller side

Follow-up questions

☐ Prove that moving the shorter side never skips the best answer.

Practise

Solve it on LeetCode: Container With Most Water (leetcode.com)

Related: Two pointers

Trapping Rain Water ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Two pointers. The most famous Hard; Google, Amazon, Goldman Sachs.

The idea

Water above bar i = min(maxLeft, maxRight) − h[i]. With two pointers, whichever side has the smaller running maximum is the bottleneck, so that side's water is already known: process it and move inward.

Steps

- l, r at ends; lmax = rmax = 0
- If h[l] < h[r]: lmax = max(lmax, h[l]); water += lmax − h[l]; l++
- Else the same on the right
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    int trap(vector<int>& h) {
        int l = 0, r = (int)h.size() - 1, lmax = 0, rmax = 0, w = 0;
        while (l < r) {
            if (h[l] < h[r]) { lmax = max(lmax, h[l]); w += lmax - h[l]; l++; }
            else { rmax = max(rmax, h[r]); w += rmax - h[r]; r--; }
        }
        return w;
    }
};
```

Common mistakes

- × Off-by-one with prefix arrays
- × Forgetting the empty-array case

Follow-up questions

- ☐ Solve it with prefix/suffix maximum arrays first, then optimise.
- ☐ Solve it with a monotonic stack.
- ☐ Trapping Rain Water II (2-D, heap + BFS)

Practise

Solve it on LeetCode: Trapping Rain Water (leetcode.com)

Related: Two pointers

L2 · INTERVIEW-READY PROBLEMS

Longest Substring Without Repeating Characters ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Sliding window. Top-5 most asked overall.

The idea

Keep a window with no repeats. Remember the last index of each character; when s[r] was seen inside the window, jump l to one past that index.

Steps

- last[c] = −1 for all c; l = 0
- For each r: if last[s[r]] ≥ l, l = last[s[r]] + 1
- last[s[r]] = r; best = max(best, r − l + 1)
- Complexity: O(n) time, O(alphabet) space

Example

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        vector<int> last(256, -1);
        int best = 0;
        for (int l = 0, r = 0; r < (int)s.size(); r++) {
            unsigned char c = s[r];
            if (last[c] >= l) l = last[c] + 1;
            last[c] = r;
            best = max(best, r - l + 1);
        }
        return best;
    }
};
```

Common mistakes

× Moving l backwards when the old index is outside the window (the ≥ l check prevents it)

Follow-up questions

☐ At most k distinct characters?

☐ Return the substring itself.

Practise

Solve it on LeetCode: Longest Substring Without Repeating Characters (leetcode.com)

Related: Sliding window

L3 · ADVANCED PROBLEMS

Minimum Window Substring ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Sliding window. Hard sliding-window favourite at Meta, LinkedIn, Uber.

The idea

Count what t needs. Expand r; when the window covers all of t (need count reaches 0), shrink l as far as possible while still covering, recording the smallest window.

Steps

- need[c] from t; missing = |t|
- Expand r: if need[s[r]] > 0, missing−−; need[s[r]]−−
- While missing == 0: record; need[s[l]]++; if it becomes > 0, missing++; l++
- Complexity: O(|s| + |t|) time, O(alphabet) space

Example

```
class Solution {
public:
    string minWindow(string s, string t) {
        vector<int> need(128, 0);
        for (char c : t) need[c]++;
        int missing = t.size(), start = 0, len = INT_MAX;
        for (int l = 0, r = 0; r < (int)s.size(); r++) {
            if (need[s[r]]-- > 0) missing--;
            while (missing == 0) {
                if (r - l + 1 < len) { len = r - l + 1; start = l; }
                if (++need[s[l++]] > 0) missing++;
            }
        }
        return len == INT_MAX ? "" : s.substr(start, len);
    }
};
```

Common mistakes

- × Forgetting duplicate letters in t
- × Recording the window after shrinking past validity

Follow-up questions

- ☐ Why does each character enter and leave at most once?

Practise

Solve it on LeetCode: Minimum Window Substring (leetcode.com)

Related: Sliding window

Subarray Sum Equals K

↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Prefix sum. Common at Meta, Google; the trap is trying a sliding window.

The idea

Negative numbers break sliding windows. A subarray (j, i] sums to k exactly when prefix[i] − prefix[j] = k. Keep counts of every prefix seen; at each i add count[prefix − k].

Steps

- count[0] = 1; s = 0
- For x: s += x; ans += count[s − k]; count[s]++
- Complexity: O(n) time, O(n) space

Example

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<long long, int> cnt{{0, 1}};
        long long s = 0; int ans = 0;
        for (int x : nums) { s += x; auto it = cnt.find(s - k); if (it ≠ cnt.end()) ans += it→second;
cnt[s]++; }
        return ans;
    }
};
```

Common mistakes

- × Missing cnt[0] = 1
- × Using a sliding window

Follow-up questions

- ☐ Longest subarray with sum k?
- ☐ Count subarrays divisible by k?

Practise

Solve it on LeetCode: Subarray Sum Equals K (leetcode.com)

Related: Prefix sums

L2 · INTERVIEW-READY PROBLEMS

Search in Rotated Sorted Array ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Binary search. Microsoft, Amazon, LinkedIn staple.

The idea

At any mid, one half [lo, mid] or [mid, hi] is sorted. Check whether the target lies in the sorted half's range; if so search there, otherwise search the other half.

Steps

- If a[lo] ≤ a[mid]: left half sorted; go left if a[lo] ≤ target < a[mid]
- Else right half sorted; go right if a[mid] < target ≤ a[hi]
- Complexity: O(log n) time, O(1) space

Example

```
class Solution {
public:
    int search(vector<int>& a, int target) {
        int lo = 0, hi = a.size() - 1;
        while (lo ⩽ hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == target) return mid;
            if (a[lo] ⩽ a[mid]) {
                if (a[lo] ⩽ target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
            } else {
                if (a[mid] < target && target ⩽ a[hi]) lo = mid + 1; else hi = mid - 1;
            }
        }
        return -1;
    }
};
```

Common mistakes

× Using < instead of ≤ in a[lo] ≤ a[mid] (fails on two elements)

Follow-up questions

☐ With duplicates? (worst case O(n): shrink lo/hi when a[lo] == a[mid] == a[hi])

☐ Find the rotation point / minimum.

Practise

Solve it on LeetCode: Search in Rotated Sorted Array (leetcode.com)

Related: Binary search

Koko Eating Bananas ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Binary search on the answer. The textbook binary-search-on-answer problem.

The idea

If Koko can finish at speed k, she can at any faster speed: feasibility is monotone. Binary search k in [1, max pile]; check(k) sums ceil(pile / k) hours.

Steps

- lo = 1, hi = max(piles)
- check(k): Σ ⌈p/k⌉ ≤ h
- Find the smallest k where check is true
- Complexity: O(n log max) time, O(1) space

Example

```
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        int lo = 1, hi = *max_element(piles.begin(), piles.end()), ans = hi;
        while (lo <= hi) {
            int k = lo + (hi - lo) / 2;
            long long hours = 0;
            for (int p : piles) hours += (p + k - 1) / k;
            if (hours <= h) { ans = k; hi = k - 1; } else lo = k + 1;
        }
        return ans;
    }
};
```

Common mistakes

- × int overflow in the hours sum
- × Starting lo at 0 (division by zero)

Follow-up questions

- ☐ Capacity to ship packages in D days.
- ☐ Split array largest sum.

Practise

Solve it on LeetCode: Koko Eating Bananas (leetcode.com)

Related: Binary search on the answer

Valid Parentheses ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Stack. The standard stack opener in first rounds.

The idea

Push opening brackets. On a closing bracket the stack top must be its matching opener; pop it. At the end the stack must be empty.

Steps

- Map) → (,] → [, } → {
- Mismatch or empty stack on a closer → false
- Complexity: O(n) time, O(n) space

Example

```
class Solution {
public:
    bool isValid(string s) {
        string st;
        for (char c : s) {
            if (c == '(' || c == '[' || c == '{') st.push_back(c);
            else {
                char want = c == ')' ? '(' : c == ']' ? '[' : '{';
                if (st.empty() || st.back() != want) return false;
                st.pop_back();
            }
        }
        return st.empty();
    }
};
```

Common mistakes

- × Forgetting the final empty check
- × Popping an empty stack

Follow-up questions

- ☐ Minimum additions to make it valid?
- ☐ Longest valid parentheses (Hard)

Practise

Solve it on LeetCode: Valid Parentheses (leetcode.com)

Related: Stacks and queues

L2 · INTERVIEW-READY PROBLEMS

Daily Temperatures ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Monotonic stack. The cleanest next-greater-element problem.

The idea

Keep a stack of indices with decreasing temperatures. When a warmer day arrives, every colder day on the stack has found its answer: pop and record the distance.

Steps

- For i: while stack top is colder than t[i], res[top] = i − top, pop
- Push i
- Complexity: O(n) time (each index pushed and popped once), O(n) space

Example

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& t) {
        vector<int> res(t.size(), 0), st;
        for (int i = 0; i < (int)t.size(); i++) {
            while (!st.empty() && t[st.back()] < t[i]) { res[st.back()] = i - st.back(); st.pop_back(); }
            st.push_back(i);
        }
        return res;
    }
};
```

Common mistakes

× Storing temperatures instead of indices

Follow-up questions

☐ Next greater element in a circular array.

Practise

Solve it on LeetCode: Daily Temperatures (leetcode.com)

Related: Monotonic stack

L3 · ADVANCED PROBLEMS

Largest Rectangle in Histogram ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Monotonic stack. Hard-round favourite at Google and Amazon; base for Maximal Rectangle.

The idea

For each bar, the widest rectangle of its height spans between its previous smaller and next smaller bars. An increasing stack finds both: when a bar is popped, the current index is its next smaller and the new stack top is its previous smaller.

Steps

- Append a 0-height sentinel
- Pop while h[top] ≥ h[i]: width = i − (new top) − 1
- Area = h[popped] × width
- Complexity: O(n) time, O(n) space

Example

```
class Solution {
public:
    int largestRectangleArea(vector<int>& h) {
        h.push_back(0);
        vector<int> st; int best = 0;
        for (int i = 0; i < (int)h.size(); i++) {
            while (!st.empty() && h[st.back()] ≥ h[i]) {
                int height = h[st.back()]; st.pop_back();
                int left = st.empty() ? -1 : st.back();
                best = max(best, height * (i - left - 1));
            }
            st.push_back(i);
        }
        h.pop_back();
        return best;
    }
};
```

Common mistakes

× Forgetting the sentinel, so bars left on the stack are never measured

Follow-up questions

☐ Maximal rectangle of 1s in a matrix (apply this per row).

Practise

Solve it on LeetCode: Largest Rectangle in Histogram (leetcode.com)

Related: Monotonic stack

L1 · CORE PROBLEMS

Reverse Linked List ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Linked list. Asked in almost every first round; often followed by the recursive version.

The idea

Walk the list re-pointing each node's next to the previous node. Save next before overwriting it.

Steps

- prev = null, cur = head
- nx = cur.next; cur.next = prev; prev = cur; cur = nx
- Return prev
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* prev = nullptr;
        while (head) { ListNode* nx = head→next; head→next = prev; prev = head; head = nx; }
        return prev;
    }
};
```

Common mistakes

× Losing the rest of the list

Follow-up questions

☐ Reverse recursively.
☐ Reverse nodes between positions m and n.
☐ Reverse in groups of k (Hard).

Practise

Solve it on LeetCode: Reverse Linked List (leetcode.com)

Related: Linked lists

Linked List Cycle II

↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Linked list. Floyd's algorithm; the proof is a common follow-up.

The idea

Slow moves 1, fast moves 2; if they meet there's a cycle. Then move one pointer to head and step both by 1: they meet at the cycle start, because head-to-start distance equals meeting-point-to-start distance (mod cycle length).

Steps

- Detect: slow/fast meet
- Reset one to head; step both by 1 until equal
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    ListNode* detectCycle(ListNode* head) {
        ListNode *s = head, *f = head;
        while (f && f->next) {
            s = s->next; f = f->next->next;
            if (s == f) { s = head; while (s != f) { s = s->next; f = f->next; } return s; }
        }
        return nullptr;
    }
};
```

Common mistakes

× Checking f->next->next without checking f->next

Follow-up questions

☐ Prove why the second phase meets at the start.

☐ Find the cycle length.

Practise

Solve it on LeetCode: Linked List Cycle II (leetcode.com)

Related: Linked lists

Merge K Sorted Lists

↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Heap. Amazon, Google, Meta Hard; tests heap use with pointers.

The idea

Push each list's head into a min-heap keyed by value. Pop the smallest, append it, push its next. Every node goes through the heap once.

Steps

- Min-heap of (val, node)
- Pop, append, push node→next
- Complexity: O(N log k) time, O(k) space

Example

```
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        auto cmp = [](ListNode* a, ListNode* b) { return a→val > b→val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
        for (auto l : lists) if (l) pq.push(l);
        ListNode dummy(0); ListNode* t = &dummy;
        while (!pq.empty()) { auto n = pq.top(); pq.pop(); t→next = n; t = n; if (n→next) pq.push(n
→next); }
        return dummy.next;
    }
};
```

Common mistakes

× Pushing null heads

Follow-up questions

☐ Divide and conquer: merge pairs (same complexity, no heap).
☐ What if lists are huge files on disk? (external merge)

Practise

Solve it on LeetCode: Merge K Sorted Lists (leetcode.com)

Related: Heaps and priority queues

L2 · INTERVIEW-READY PROBLEMS

LRU Cache ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Linked list. One of the most-asked design questions for freshers at Amazon, Microsoft, Swiggy.

The idea

Need O(1) get and put with recency order. A hash map gives O(1) lookup of a node; a doubly linked list gives O(1) move-to-front and remove-from-back. std::list + splice does both.

Steps

- map key → list iterator
- get: splice the node to the front
- put: update or insert at front; if over capacity, erase the back and its map entry
- Complexity: O(1) per operation, O(capacity) space

Example

```
class LRUCache {
    int cap;
    list<pair<int,int>> dq;                                  // front = most recent
    unordered_map<int, list<pair<int,int>>::iterator> pos;
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        auto it = pos.find(key);
        if (it == pos.end()) return -1;
        dq.splice(dq.begin(), dq, it->second);
        return it->second->second;
    }
    void put(int key, int value) {
        auto it = pos.find(key);
        if (it != pos.end()) { it->second->second = value; dq.splice(dq.begin(), dq, it->second); return; }
        dq.emplace_front(key, value); pos[key] = dq.begin();
        if ((int)dq.size() > cap) { pos.erase(dq.back().first); dq.pop_back(); }
    }
};
```

Common mistakes

- × Forgetting to move on update
- × Erasing from the map after popping (dangling)

Follow-up questions

- ☐ Make it thread-safe.
- ☐ LFU cache (Hard).
- ☐ How does Redis approximate LRU?

Practise

Solve it on LeetCode: LRU Cache (leetcode.com)

Related: Linked lists

Binary Tree Level Order Traversal ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: BFS. Base for zigzag, right view, and many tree BFS questions.

The idea

BFS with a queue; process exactly the current queue size each round to keep levels separate.

Steps

- q = [root]
- While q: sz = q.size(); pop sz nodes into one level, pushing children
- Complexity: O(n) time, O(width) space

Example

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> res; if (!root) return res;
        queue<TreeNode*> q; q.push(root);
        while (!q.empty()) {
            int sz = q.size(); res.emplace_back();
            while (sz--) { auto n = q.front(); q.pop(); res.back().push_back(n->val);
                if (n->left) q.push(n->left); if (n->right) q.push(n->right); }
        }
        return res;
    }
};
```

Common mistakes

× Reading q.size() inside the loop condition (it changes)

Follow-up questions

☐ Zigzag order.
☐ Right side view.
☐ Average of each level.

Practise

Solve it on LeetCode: Binary Tree Level Order Traversal (leetcode.com)

Related: BFS and graph basics

L2 · INTERVIEW-READY PROBLEMS

Lowest Common Ancestor of a Binary Tree ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Trees. Asked at Meta, Amazon, Microsoft; short code, tricky reasoning.

The idea

Recurse: return the node if it's p or q. If the left and right subtrees each return non-null, this node is the LCA; otherwise pass up whichever side is non-null.

Steps

- Base: null, p or q → return it
- L = lca(left), R = lca(right)
- Both non-null → root; else the non-null one
- Complexity: O(n) time, O(h) space

Example

```
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (!root || root == p || root == q) return root;
        TreeNode* L = lowestCommonAncestor(root->left, p, q);
        TreeNode* R = lowestCommonAncestor(root->right, p, q);
        return L && R ? root : (L ? L : R);
    }
};
```

Common mistakes

× Assuming it's a BST (that version walks by value)

Follow-up questions

☐ LCA in a BST.

☐ What if p or q may not exist in the tree?

Practise

Solve it on LeetCode: Lowest Common Ancestor of a Binary Tree (leetcode.com)

Related: Binary trees

Validate Binary Search Tree ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: BST. Tests whether you know the BST property is global, not parent-child.

The idea

Every node must lie strictly between bounds inherited from its ancestors. Pass (lo, hi) down: left child gets (lo, val), right child gets (val, hi). Use long long or pointers for the bounds.

Steps

- valid(node, lo, hi): lo < val < hi
- Recurse left with hi = val, right with lo = val
- Complexity: O(n) time, O(h) space

Example

```
class Solution {
    bool ok(TreeNode* n, long long lo, long long hi) {
        if (!n) return true;
        if (n->val <= lo || n->val >= hi) return false;
        return ok(n->left, lo, n->val) && ok(n->right, n->val, hi);
    }
public:
    bool isValidBST(TreeNode* root) { return ok(root, LLONG_MIN, LLONG_MAX); }
};
```

Common mistakes

- × Only comparing a node with its children
- × int bounds failing for INT_MIN/INT_MAX values

Follow-up questions

- ☐ Do it with an inorder traversal (must be strictly increasing).
- ☐ Recover a BST with two swapped nodes.

Practise

Solve it on LeetCode: Validate Binary Search Tree (leetcode.com)

Related: Binary search trees

L3 · ADVANCED PROBLEMS

Binary Tree Maximum Path Sum ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Trees. Google and Meta Hard; the 'return one thing, record another' pattern.

The idea

A path can bend at one node. For each node, the best path through it is val + best downward gain from left + from right (ignoring negative gains). But only one side can be returned to the parent.

Steps

- gain(n) = n.val + max(0, gain(left), gain(right))... returning only one side
- best = max(best, val + max(0,L) + max(0,R))
- Complexity: O(n) time, O(h) space

Example

```
class Solution {
    int best = INT_MIN;
    int gain(TreeNode* n) {
        if (!n) return 0;
        int L = max(0, gain(n→left)), R = max(0, gain(n→right));
        best = max(best, n→val + L + R);
        return n→val + max(L, R);
    }
public:
    int maxPathSum(TreeNode* root) { gain(root); return best; }
};
```

Common mistakes

- × Returning L + R to the parent (that's not a path)
- × Initialising best to 0 (fails on all-negative trees)

Follow-up questions

- ☐ Diameter of a binary tree (same pattern with edge counts).

Practise

Solve it on LeetCode: Binary Tree Maximum Path Sum (leetcode.com)

Related: Binary trees

Number of Islands ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: DFS. Most-asked graph problem; grid as an implicit graph.

The idea

Scan the grid; each unvisited '1' starts a new island. Flood-fill it (DFS or BFS) and mark visited so it isn't counted again.

Steps

- For each cell == '1': count++, flood fill
- Fill: mark '0', recurse into 4 neighbours inside bounds
- Complexity: O(rows × cols) time and space (recursion)

Example

```
class Solution {
    void fill(vector<vector<char>>& g, int r, int c) {
        if (r < 0 || c < 0 || r ≥ (int)g.size() || c ≥ (int)g[0].size() || g[r][c] ≠ '1') return;
        g[r][c] = '0';
        fill(g, r + 1, c); fill(g, r - 1, c); fill(g, r, c + 1); fill(g, r, c - 1);
    }
public:
    int numIslands(vector<vector<char>>& g) {
        int n = 0;
        for (int r = 0; r < (int)g.size(); r++) for (int c = 0; c < (int)g[0].size(); c++) if (g[r][c] ==
'1') { n++; fill(g, r, c); }
        return n;
    }
};
```

Common mistakes

- × Deep recursion on big grids (use BFS or an explicit stack)
- × Not mentioning that you modify the input

Follow-up questions

- ☐ Max area of an island.
- ☐ Number of distinct islands.
- ☐ Do it with DSU.

Practise

Solve it on LeetCode: Number of Islands (leetcode.com)

Related: DFS, components and cycles

Course Schedule ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Topological sort. Amazon and Google; cycle detection in a directed graph.

The idea

Courses are nodes, prerequisites are edges. All courses can be finished exactly when the graph has no cycle. Kahn's algorithm: repeatedly take nodes with in-degree 0; if you process all n, there's no cycle.

Steps

- Build adjacency and in-degrees
- Queue all in-degree-0 nodes; pop, decrement neighbours
- Return processed == n
- Complexity: O(V + E) time and space

Example

```
class Solution {
public:
    bool canFinish(int n, vector<vector<int>>& pre) {
        vector<vector<int>> adj(n); vector<int> indeg(n, 0);
        for (auto& e : pre) { adj[e[1]].push_back(e[0]); indeg[e[0]]++; }
        queue<int> q; for (int i = 0; i < n; i++) if (!indeg[i]) q.push(i);
        int done = 0;
        while (!q.empty()) { int u = q.front(); q.pop(); done++; for (int v : adj[u]) if (--indeg[v] == 0)
q.push(v); }
        return done == n;
    }
};
```

Common mistakes

× Reversing the edge direction (harmless here, wrong in Course Schedule II's order)

Follow-up questions

☐ Return a valid order (Course Schedule II).
☐ Do it with DFS colouring.

Practise

Solve it on LeetCode: Course Schedule (leetcode.com)

Related: Topological sort

Rotting Oranges ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: BFS. Multi-source BFS; Amazon's favourite grid BFS.

The idea

All rotten oranges start the BFS together. Each BFS level is one minute. At the end, if any fresh orange remains, return −1.

Steps

- Queue all rotten cells; count fresh
- BFS level by level, rotting neighbours and decrementing fresh
- Answer = levels used, or −1
- Complexity: O(rows × cols) time and space

Example

```
class Solution {
public:
    int orangesRotting(vector<vector<int>>& g) {
        int R = g.size(), C = g[0].size(), fresh = 0, mins = 0;
        queue<pair<int,int>> q;
        for (int r = 0; r < R; r++) for (int c = 0; c < C; c++) { if (g[r][c] == 2) q.push({r, c}); else if
(g[r][c] == 1) fresh++; }
        int dr[] = {1,-1,0,0}, dc[] = {0,0,1,-1};
        while (!q.empty() && fresh) {
            for (int sz = q.size(); sz--; ) {
                auto [r, c] = q.front(); q.pop();
                for (int k = 0; k < 4; k++) { int nr = r + dr[k], nc = c + dc[k];
                    if (nr >= 0 && nc >= 0 && nr < R && nc < C && g[nr][nc] == 1) { g[nr][nc] = 2; fresh--;
q.push({nr, nc}); } }
            }
            mins++;
        }
        return fresh ? -1 : mins;
    }
};
```

Common mistakes

× Counting a minute after the last level (stop when fresh is 0)

Follow-up questions

☐ 01 Matrix (distance to nearest 0).

☐ Walls and gates.

Practise

Solve it on LeetCode: Rotting Oranges (leetcode.com)

Related: BFS and graph basics

Network Delay Time

↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Dijkstra. The standard Dijkstra question.

The idea

Shortest time for the signal to reach every node = the largest shortest-path distance from k. Run Dijkstra with a min-heap; if any node is unreachable, return −1.

Steps

- dist[k] = 0; heap (d, node)
- Pop; skip stale; relax edges
- Answer = max(dist) or −1
- Complexity: O((V + E) log V) time

Example

```
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int,int>>> adj(n + 1);
        for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});
        vector<int> d(n + 1, INT_MAX); d[k] = 0;
        priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq; pq.push({0, k});
        while (!pq.empty()) {
            auto [du, u] = pq.top(); pq.pop();
            if (du > d[u]) continue;
            for (auto [v, w] : adj[u]) if (du + w < d[v]) { d[v] = du + w; pq.push({d[v], v}); }
        }
        int ans = *max_element(d.begin() + 1, d.end());
        return ans == INT_MAX ? -1 : ans;
    }
};
```

Common mistakes

× Not skipping stale heap entries (still correct, slower)
× 1-indexed nodes

Follow-up questions

☐ Cheapest flights within K stops (Bellman-Ford / BFS by stops).
☐ Path with minimum effort.

Practise

Solve it on LeetCode: Network Delay Time (leetcode.com)

Related: Dijkstra and shortest paths

Number of Provinces ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: DSU. Connected components; a clean place to show Union-Find.

The idea

Each city starts alone. For every connected pair, union their sets; each successful union reduces the count of components by one.

Steps

- parent[i] = i; comps = n
- For i < j with isConnected[i][j]: if unite(i, j), comps−−
- Complexity: O(n² α(n)) time, O(n) space

Example

```
class Solution {
    vector<int> p;
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
public:
    int findCircleNum(vector<vector<int>>& g) {
        int n = g.size(), comps = n; p.resize(n); iota(p.begin(), p.end(), 0);
        for (int i = 0; i < n; i++) for (int j = i + 1; j < n; j++)
            if (g[i][j]) { int a = find(i), b = find(j); if (a != b) { p[a] = b; comps--; } }
        return comps;
    }
};
```

Common mistakes

× Comparing raw parents instead of roots

Follow-up questions

☐ Solve it with DFS too.
☐ Redundant connection.
☐ Accounts merge.

Practise

Solve it on LeetCode: Number of Provinces (leetcode.com)

Related: Disjoint set union (Union-Find)

L2 · INTERVIEW-READY PROBLEMS

House Robber ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: DP. The first DP most interviewers ask.

The idea

At house i you either rob it (take a[i] + best up to i−2) or skip it (best up to i−1). Only the last two values are needed.

Steps

- dp[i] = max(dp[i−1], dp[i−2] + a[i])
- Keep two variables
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    int rob(vector<int>& a) {
        int prev2 = 0, prev1 = 0;
        for (int x : a) { int cur = max(prev1, prev2 + x); prev2 = prev1; prev1 = cur; }
        return prev1;
    }
};
```

Common mistakes

× Greedy 'take every other house'

Follow-up questions

☐ Houses in a circle (House Robber II).
☐ Houses on a tree (House Robber III).

Practise

Solve it on LeetCode: House Robber (leetcode.com)

Related: Dynamic programming

Coin Change ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Knapsack. Most-asked unbounded-knapsack problem; greedy fails on it.

The idea

dp[x] = fewest coins to make x. dp[x] = min over coins c ≤ x of dp[x − c] + 1. Greedy fails (coins 1, 3, 4 for 6: greedy gives 3, best is 2).

Steps

- dp[0] = 0, others = ∞
- For x in 1..amount, for c: dp[x] = min(dp[x], dp[x−c] + 1)
- Answer dp[amount] or −1
- Complexity: O(amount × coins) time, O(amount) space

Example

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, INT_MAX); dp[0] = 0;
        for (int x = 1; x ⩽ amount; x++)
            for (int c : coins) if (c ⩽ x && dp[x - c] ≠ INT_MAX) dp[x] = min(dp[x], dp[x - c] + 1);
        return dp[amount] == INT_MAX ? -1 : dp[amount];
    }
};
```

Common mistakes

- × INT_MAX + 1 overflow
- × Using greedy

Follow-up questions

- ☐ Count the number of ways (Coin Change II: coins in the outer loop).
- ☐ Print the coins used.

Practise

Solve it on LeetCode: Coin Change (leetcode.com)

Related: Knapsack and subset DP

L2 · INTERVIEW-READY PROBLEMS

Longest Increasing Subsequence ↑ Index

WHY IT MATTERS FOR YOU

Pattern: DP. Asked for both the O(n²) DP and the O(n log n) follow-up.

The idea

O(n²): dp[i] = 1 + max dp[j] for j < i with a[j] < a[i]. O(n log n): keep tails[len] = smallest possible tail of an increasing subsequence of length len+1; for each x, replace the first tail ≥ x (lower_bound) or append.

Steps

- tails = []
- For x: it = lower_bound(tails, x); replace or push_back
- Answer = tails.size()
- Complexity: O(n log n) time, O(n) space

Example

```
class Solution {
public:
    int lengthOfLIS(vector<int>& a) {
        vector<int> tails;
        for (int x : a) {
            auto it = lower_bound(tails.begin(), tails.end(), x);
            if (it == tails.end()) tails.push_back(x); else *it = x;
        }
        return tails.size();
    }
};
```

Common mistakes

- × Thinking tails is the subsequence itself (it isn't)
- × upper_bound for strictly increasing (use lower_bound)

Follow-up questions

- ☐ Print the actual subsequence.
- ☐ Russian doll envelopes (Hard).

Practise

Solve it on LeetCode: Longest Increasing Subsequence (leetcode.com)

Related: Dynamic programming

Longest Common Subsequence ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: String DP. The base of all two-string DP.

The idea

dp[i][j] = LCS of a[0..i) and b[0..j). If a[i−1] == b[j−1], dp[i][j] = dp[i−1][j−1] + 1; else max(dp[i−1][j], dp[i][j−1]).

Steps

- (n+1) × (m+1) table, row/column 0 are 0
- Fill row by row
- Space-optimise to two rows
- Complexity: O(n × m) time, O(m) space

Example

```
class Solution {
public:
    int longestCommonSubsequence(string a, string b) {
        int n = a.size(), m = b.size();
        vector<int> prev(m + 1, 0), cur(m + 1, 0);
        for (int i = 1; i ⩽ n; i++) {
            for (int j = 1; j ⩽ m; j++)
                cur[j] = a[i - 1] == b[j - 1] ? prev[j - 1] + 1 : max(prev[j], cur[j - 1]);
            swap(prev, cur);
        }
        return prev[m];
    }
};
```

Common mistakes

× Mixing string index i−1 with dp index i

Follow-up questions

☐ Print the LCS.
☐ Shortest common supersequence.
☐ Longest palindromic subsequence.

Practise

Solve it on LeetCode: Longest Common Subsequence (leetcode.com)

Related: String DP: LCS and edit distance

Edit Distance

↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: String DP. Google, Amazon, Microsoft; real-world use in spell checkers and diff.

The idea

dp[i][j] = edits to turn a[0..i) into b[0..j). Equal last characters cost nothing (diagonal). Otherwise 1 + min(insert dp[i][j−1], delete dp[i−1][j], replace dp[i−1][j−1]). Base: dp[i][0] = i, dp[0][j] = j.

Steps

- Fill base row and column
- Apply the transition
- Answer dp[n][m]
- Complexity: O(n × m) time and space (O(m) with two rows)

Example

```
class Solution {
public:
    int minDistance(string a, string b) {
        int n = a.size(), m = b.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1));
        for (int i = 0; i ⩽ n; i++) dp[i][0] = i;
        for (int j = 0; j ⩽ m; j++) dp[0][j] = j;
        for (int i = 1; i ⩽ n; i++) for (int j = 1; j ⩽ m; j++)
            dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1] : 1 + min({dp[i][j - 1], dp[i - 1][j], dp[i
- 1][j - 1]});
        return dp[n][m];
    }
};
```

Common mistakes

× Wrong base cases

Follow-up questions

☐ Reduce to O(min(n, m)) space.

☐ One edit distance (O(n)).

Practise

Solve it on LeetCode: Edit Distance (leetcode.com)

Related: String DP: LCS and edit distance

Partition Equal Subset Sum ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Knapsack. 0/1 knapsack in disguise; Amazon and Meta.

The idea

Equal halves exist iff some subset sums to total/2 (total must be even). Boolean 0/1 knapsack over sums, iterating sums downwards so each number is used once.

Steps

- If total is odd → false
- dp[0] = true; for x: for s = T..x: dp[s] |= dp[s − x]
- Complexity: O(n × total) time, O(total) space

Example

```
class Solution {
public:
    bool canPartition(vector<int>& a) {
        int total = accumulate(a.begin(), a.end(), 0);
        if (total % 2) return false;
        int T = total / 2; vector<char> dp(T + 1, 0); dp[0] = 1;
        for (int x : a) for (int s = T; s ⩾ x; s--) dp[s] ⊨ dp[s - x];
        return dp[T];
    }
};
```

Common mistakes

× Iterating sums upwards (reuses a number)

Follow-up questions

☐ Use a bitset for speed.

☐ Target sum (count sign assignments).

Practise

Solve it on LeetCode: Partition Equal Subset Sum (leetcode.com)

Related: Knapsack and subset DP

Subsets

↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Backtracking. The entry point to backtracking; often followed by Subsets II.

The idea

Each element is either in or out: a binary decision tree with 2^n leaves. Recurse with pick / skip, or iterate: for each element, copy every existing subset and add it.

Steps

- go(i): if i == n, record
- Pick a[i], go(i+1), unpick; go(i+1)
- Complexity: O(n × 2^n) time, O(n) recursion depth

Example

```
class Solution {
    vector<vector<int>> res; vector<int> cur;
    void go(vector<int>& a, int i) {
        if (i == (int)a.size()) { res.push_back(cur); return; }
        cur.push_back(a[i]); go(a, i + 1); cur.pop_back();
        go(a, i + 1);
    }
public:
    vector<vector<int>> subsets(vector<int>& a) { go(a, 0); return res; }
};
```

Common mistakes

× Forgetting to pop after the pick branch

Follow-up questions

☐ Subsets II with duplicates (sort, skip equal siblings).

☐ Do it with bitmasks.

Practise

Solve it on LeetCode: Subsets (leetcode.com)

Related: Recursion and backtracking

Merge Intervals ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Intervals. Top-asked at Google, Meta, Bloomberg.

The idea

Sort by start. Walk once: if the current interval starts before the last merged one ends, extend the end; otherwise start a new merged interval.

Steps

- Sort by start
- If start ≤ last.end: last.end = max(last.end, end); else push
- Complexity: O(n log n) time, O(n) output

Example

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& iv) {
        sort(iv.begin(), iv.end());
        vector<vector<int>> res;
        for (auto& x : iv) {
            if (!res.empty() && x[0] ≤ res.back()[1]) res.back()[1] = max(res.back()[1], x[1]);
            else res.push_back(x);
        }
        return res;
    }
};
```

Common mistakes

× Setting end = x.end instead of max (a long interval swallowing a short one)

Follow-up questions

☐ Insert interval.
☐ Meeting rooms II (minimum rooms).

Practise

Solve it on LeetCode: Merge Intervals (leetcode.com)

Related: Interval problems

L2 · INTERVIEW-READY PROBLEMS

Kth Largest Element in an Array ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Heap. Asked with the follow-up 'can you beat O(n log n)?'.

The idea

Keep a min-heap of size k: its top is the k-th largest. Quickselect averages O(n).

Steps

- Push each x; pop when size > k
- Return top
- Complexity: O(n log k) time, O(k) space

Example

```
class Solution {
public:
    int findKthLargest(vector<int>& a, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for (int x : a) { pq.push(x); if ((int)pq.size() > k) pq.pop(); }
        return pq.top();
    }
};
```

Common mistakes

× Max-heap of everything (O(n log n))

Follow-up questions

☐ Quickselect (average O(n), worst $O(n^2)$).

☐ nth_element in C++.

Practise

Solve it on LeetCode: Kth Largest Element in an Array (leetcode.com)

Related: Heaps and priority queues

Implement Trie (Prefix Tree) ↑ Index

WHY IT MATTERS FOR YOU

Pattern: Trie. Common design-style question; base for autocomplete and Word Search II.

The idea

Each node has 26 children and an end-of-word flag. Insert walks/creates nodes; search walks and checks the flag; startsWith only walks.

Steps

- insert: create missing children, mark end
- search: walk, return end flag
- startsWith: walk, return true
- Complexity: O(length) per operation, O(total characters × 26) space

Example

```
class Trie {
    struct Node { Node* ch[26] = {}; bool end = false; };
    Node* root = new Node();
    Node* walk(const string& w) { Node* n = root; for (char c : w) { n = n->ch[c - 'a']; if (!n) return
nullptr; } return n; }
public:
    void insert(string w) { Node* n = root; for (char c : w) { if (!n->ch[c - 'a']) n->ch[c - 'a'] = new
Node(); n = n->ch[c - 'a']; } n->end = true; }
    bool search(string w) { Node* n = walk(w); return n && n->end; }
    bool startsWith(string p) { return walk(p) != nullptr; }
};
```

Common mistakes

× Leaking memory (mention a destructor or arena)

Follow-up questions

☐ Word Search II.

☐ Autocomplete top-3 suggestions.

☐ Max XOR with a bit trie.

Practise

Solve it on LeetCode: Implement Trie (Prefix Tree) (leetcode.com)

Related: Tries

L2 · INTERVIEW-READY PROBLEMS

Jump Game ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Greedy. A short greedy problem that tests whether you'd reach for DP unnecessarily.

The idea

Track the furthest index reachable so far. If you ever stand on an index beyond it, you're stuck.

Steps

- reach = 0
- For i: if i > reach → false; reach = max(reach, i + a[i])
- Complexity: O(n) time, O(1) space

Example

```
class Solution {
public:
    bool canJump(vector<int>& a) {
        int reach = 0;
        for (int i = 0; i < (int)a.size(); i++) { if (i > reach) return false; reach = max(reach, i +
a[i]); }
        return true;
    }
};
```

Common mistakes

× $O(n^2)$ DP when greedy suffices

Follow-up questions

☐ Minimum number of jumps (Jump Game II, greedy BFS by levels).

Practise

Solve it on LeetCode: Jump Game (leetcode.com)

Related: Greedy algorithms

Find Median from Data Stream ↑ Index

> **WHY IT MATTERS FOR YOU**
> Pattern: Heap. Classic two-heap design; Google, Amazon, Microsoft.

The idea

Keep the smaller half in a max-heap and the larger half in a min-heap, sizes equal or the max-heap one bigger. The median is the max-heap top, or the average of both tops.

Steps

- Push to lo (max-heap), move lo's top to hi
- If hi is bigger, move hi's top back to lo
- Median from the tops
- Complexity: O(log n) per add, O(1) median

Example

```
class MedianFinder {
    priority_queue<int> lo;                                  // max-heap: smaller half
    priority_queue<int, vector<int>, greater<int>> hi;       // min-heap: larger half
public:
    void addNum(int x) {
        lo.push(x); hi.push(lo.top()); lo.pop();
        if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
    }
    double findMedian() { return lo.size() > hi.size() ? lo.top() : (lo.top() + (double)hi.top()) / 2.0; }
};
```

Common mistakes

- × Integer division when averaging
- × int overflow when adding two big tops (cast first)

Follow-up questions

- ☐ If all numbers are in 0..100? (counting array)
- ☐ Sliding window median.

Practise

Solve it on LeetCode: Find Median from Data Stream (leetcode.com)

Related: Heaps and priority queues

L1 · CORE CORE CS

Processes and threads ↑ Index

> **WHY IT MATTERS FOR YOU**
> The most-asked OS question in every technical round, product or service company.

The concept

A process is a running program with its own address space (code, data, heap, stack), file descriptors and a PCB (process control block) in the kernel.

A thread is a unit of execution inside a process. Threads share the process's code, data, heap and files, but each has its own stack, registers and program counter.

Threads are cheaper because creating one and switching between threads of the same process don't need a new address space or a TLB flush. The cost of sharing is that one thread's bug (or a missing lock) can corrupt the whole process.

Key points

- Process states: new, ready, running, waiting, terminated
- fork() copies the process (copy-on-write); exec() replaces its image
- User-level vs kernel-level threads
- Zombie: finished but parent hasn't called wait(); orphan: parent died first

Common mistakes

× Saying threads share the stack (they don't)

Interview questions

☐ Process vs thread, and why threads are cheaper?
☐ What happens on a context switch?
☐ What is a zombie process?

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · GfG: process vs thread (geeksforgeeks.org) · Gate Smashers (YouTube) (youtube.com)

Related: CPU scheduling · Synchronisation: mutex, semaphore, race conditions · Memory management and virtual memory

L1 · CORE CORE CS

CPU scheduling ↑ Index

> **WHY IT MATTERS FOR YOU**
> Numerical questions in OAs (compute waiting time) and conceptual ones in interviews.

The concept

The scheduler picks which ready process runs next. Goals: high CPU use and throughput, low waiting, turnaround and response time, fairness.

FCFS (convoy effect), SJF/SRTF (optimal average waiting, needs burst estimates, can starve long jobs), priority (starvation fixed by ageing), round robin (time quantum; good response time), multilevel feedback queues (what real systems approximate). Linux uses CFS, which tracks virtual runtime in a red-black tree.

Key points

- Turnaround = completion − arrival; waiting = turnaround − burst
- Preemptive vs non-preemptive
- Quantum too small → too many context switches; too big → FCFS

Common mistakes

× Mixing up turnaround and response time

Interview questions

☐ Compare round robin and SJF.

☐ What is starvation and how does ageing fix it?

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · Gate Smashers (YouTube) (youtube.com)

Related: Processes and threads

L1 · CORE CORE CS

Deadlock ↑ Index

> **WHY IT MATTERS FOR YOU**
> Top-5 OS question. Also asked in DBMS (transaction deadlocks).

The concept

A set of processes each waiting for a resource held by another in the set, so none can proceed.

Coffman conditions, all four needed: mutual exclusion, hold and wait, no preemption, circular wait.

Handling: prevention (break one condition, e.g. always lock resources in a fixed global order to stop circular wait), avoidance (Banker's algorithm keeps the system in a safe state), detection + recovery (wait-for graph, kill or roll back), or ignore it (most OSes).

Key points

- Fixed lock ordering is the practical fix in code
- Livelock: processes keep changing state but make no progress
- Starvation ≠ deadlock

Common mistakes

× Listing only three conditions

Interview questions

☐ Four conditions of deadlock?
☐ How would you prevent deadlock in your code?
☐ Explain Banker's algorithm.

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · GfG: deadlock (geeksforgeeks.org) · Gate Smashers (YouTube) (youtube.com)

Related: Synchronisation: mutex, semaphore, race conditions · Transactions, ACID and isolation

L2 · INTERVIEW-READY CORE CS

Synchronisation: mutex, semaphore, race conditions ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked in OS rounds and C++/HFT interviews; producer-consumer is a classic whiteboard question.

The concept

A race condition happens when the result depends on the timing of threads touching shared data. The code that touches it is the critical section; it needs mutual exclusion, progress and bounded waiting.

A mutex is a lock with an owner: only the thread that locked it unlocks it. A semaphore is a counter: wait() decrements and blocks at 0, signal() increments. A binary semaphore looks like a mutex but has no owner; a counting semaphore guards N identical resources.

Producer-consumer with a bounded buffer: semaphores empty = N and full = 0, plus a mutex for the buffer. Condition variables let a thread sleep until a predicate becomes true (always wait in a loop).

Key points

- Spinlock: busy-waits, good for very short sections
- Atomics avoid locks for single variables
- Readers-writers and dining philosophers are the other classics

Code

```
std::mutex m; std::condition_variable cv; std::queue<int> q;
void produce(int x) { { std::lock_guard<std::mutex> g(m); q.push(x); } cv.notify_one(); }
int consume() { std::unique_lock<std::mutex> g(m); cv.wait(g, [] { return !q.empty(); });
    int x = q.front(); q.pop(); return x; }
```

Common mistakes

× Using if instead of while around a condition wait (spurious wake-ups)

Interview questions

- ☐ Mutex vs semaphore with an example.
- ☐ Write producer-consumer.
- ☐ What is a race condition?

Learn from

OSTEP (free OS book) (pages.cs.wisc.edu) · cppreference: thread support (en.cppreference.com)

Related: Deadlock · Processes and threads

L2 · INTERVIEW-READY CORE CS

Memory management and virtual memory ↑ Index

> **WHY IT MATTERS FOR YOU**
> Second most-asked OS area after processes. Also needed for binary exploitation (stack, heap, ASLR).

The concept

Each process sees its own virtual address space. The MMU translates virtual pages to physical frames using page tables; the TLB caches recent translations.

Why: isolation (one process can't read another's memory), programs bigger than RAM, simpler loading, and sharing (shared libraries, copy-on-write after fork).

A page fault happens when a page isn't in RAM: the OS loads it from disk and may evict another (FIFO, LRU, optimal; Belady's anomaly hits FIFO). Thrashing is when the system spends more time paging than running because working sets don't fit.

Key points

- Internal fragmentation (paging) vs external (segmentation)
- Multi-level page tables save memory
- Effective access time with TLB hit ratio

Common mistakes

× Confusing virtual memory with swap space only

Interview questions

☐ Why do we need virtual memory?
☐ What happens on a page fault?
☐ What is thrashing?
☐ LRU page replacement: implement it (→ LRU cache).

Learn from

OSTEP (free OS book) `(pages.cs.wisc.edu)` · Gate Smashers (YouTube) `(youtube.com)`

Related: Processes and threads · Binary exploitation basics

L1 · CORE CORE CS

Normalisation and keys ↑ Index

> **WHY IT MATTERS FOR YOU**
> The standard DBMS opener in interviews and a frequent OA MCQ.

The concept

Keys: a super key identifies a row; a candidate key is a minimal super key; the primary key is the chosen candidate; a foreign key references another table's key.

Normalisation removes redundancy that causes insert, update and delete anomalies. 1NF: atomic values. 2NF: 1NF and no partial dependency on part of a composite key. 3NF: 2NF and no transitive dependency (non-key → non-key). BCNF: every determinant is a candidate key.

Denormalisation trades redundancy for faster reads, common in analytics and NoSQL.

Key points

- Example: Student(roll, name, dept, dept_head) → dept_head depends on dept → split into Student and Dept for 3NF
- Lossless join and dependency preservation
- ER diagram → tables

Common mistakes

× Not giving a concrete table example

Interview questions

☐ Explain 1NF to 3NF with one table.
☐ Difference between 3NF and BCNF?
☐ Primary vs unique key?

Learn from

GfG: normal forms (geeksforgeeks.org) · Gate Smashers (YouTube) (youtube.com) · CMU 15-445 (free lectures) (15445.courses.cs.cmu.edu)

Related: Transactions, ACID and isolation · Indexing and B+ trees · SQL for interviews

L2 · INTERVIEW-READY CORE CS

Transactions, ACID and isolation ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked at every product company, especially fintechs like Razorpay where money moves.

The concept

Atomicity (all or nothing, via undo logs), Consistency (constraints hold before and after), Isolation (concurrent transactions behave as if run in some order), Durability (committed data survives crashes, via write-ahead logging).

Anomalies: dirty read, non-repeatable read, phantom read. Isolation levels: Read Uncommitted (allows all), Read Committed (stops dirty reads), Repeatable Read (also stops non-repeatable reads), Serializable (stops phantoms).

Implementations use locks (two-phase locking) or MVCC (each transaction reads a snapshot; PostgreSQL, MySQL InnoDB).

Key points

- Read Committed prevents dirty reads
- 2PL: growing phase then shrinking phase
- Deadlocks between transactions are detected and one is rolled back

Common mistakes

× Saying 'Consistency' means the same as in CAP (it doesn't)

Interview questions

☐ Explain ACID with a bank transfer.
☐ Which isolation level prevents dirty reads?
☐ What is MVCC?

Learn from

GfG: ACID (geeksforgeeks.org) · CMU 15-445 (15445.courses.cs.cmu.edu)

Related: Normalisation and keys · Deadlock · CAP theorem and consistency

L2 · INTERVIEW-READY CORE CS

Indexing and B+ trees ↑ Index

> **WHY IT MATTERS FOR YOU**
> 'Why is my query slow?' is a real-job question and a favourite follow-up in backend interviews.

The concept

An index is a separate structure that maps column values to row locations so the database avoids scanning the whole table.

Most databases use B+ trees: wide, shallow trees (each node holds hundreds of keys, sized to a disk page) with all values in linked leaves. A lookup touches about 3–4 pages for millions of rows, and range scans walk the leaves.

A clustered index stores rows in key order (one per table, usually the primary key); secondary indexes point to it. Composite indexes follow the leftmost-prefix rule.

Key points

- Indexes speed reads but slow writes and use space
- Covering index answers a query from the index alone
- Hash indexes: equality only, no ranges
- Use EXPLAIN to see the query plan

Common mistakes

- × Indexing low-cardinality columns (gender) expecting gains
- × Applying a function to an indexed column in WHERE stops index use

Interview questions

- ☐ How does an index speed up a query?
- ☐ Why B+ tree and not a binary tree or hash?
- ☐ Clustered vs non-clustered?

Learn from

Use The Index, Luke (free) `(use-the-index-luke.com)` · CMU 15-445 `(15445.courses.cs.cmu.edu)`

Related: Normalisation and keys · SQL for interviews

L1 · CORE CORE CS

SQL for interviews ↑ Index

> **WHY IT MATTERS FOR YOU**
> Many OAs include 1–2 SQL questions; analyst-flavoured and backend roles always do.

The concept

Order of evaluation: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. WHERE filters rows, HAVING filters groups.

Joins: INNER (matches only), LEFT (all left rows), self-join (employee–manager), anti-join (LEFT JOIN … WHERE right.id IS NULL).

Window functions compute over a partition without collapsing rows: ROW_NUMBER, RANK, DENSE_RANK, LAG/LEAD, running SUM() OVER (ORDER BY …).

Key points

- DENSE_RANK for 'Nth highest' with ties
- COUNT(col) ignores NULLs, COUNT(*) doesn't
- Parameterised queries prevent SQL injection

Code

```
SELECT dept, name, salary FROM (
  SELECT d.name AS dept, e.name, e.salary,
         DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS r
  FROM employee e JOIN department d ON d.id = e.dept_id
) t WHERE r = 2;
```

Common mistakes

- × Using WHERE with an aggregate
- × Forgetting NULL handling in comparisons

Interview questions

- ☐ Second-highest salary per department.
- ☐ Find duplicate emails.
- ☐ Difference between RANK and DENSE_RANK?

Practise

LeetCode SQL 50 (leetcode.com) · SQLBolt (sqlbolt.com)

Learn from

SQLBolt (sqlbolt.com) · Mode SQL tutorial (mode.com)

Related: Indexing and B+ trees · SQL injection

L1 · CORE CORE CS

OSI and TCP/IP models ↑ Index

> **WHY IT MATTERS FOR YOU**
> The first Computer Networks question in most interviews, and the vocabulary for everything in security.

The concept

OSI has 7 layers: Physical, Data link (MAC, switches), Network (IP, routers), Transport (TCP/UDP, ports), Session, Presentation, Application. TCP/IP merges them into 4: Link, Internet, Transport, Application.

Each layer wraps the one above with its own header (encapsulation): HTTP data → TCP segment → IP packet → Ethernet frame.

Key points

- Devices: hub (L1), switch (L2), router (L3)
- ARP maps IP → MAC
- Subnetting: /24 = 256 addresses, 254 usable hosts

Common mistakes

× Putting TLS or HTTP at the wrong layer without explaining

Interview questions

☐ Explain the OSI model.
☐ Which layer does a router work at?
☐ What is ARP?

Learn from

Gate Smashers (YouTube) (youtube.com) · Computer Networking: A Top-Down Approach (Kurose) (gaia.cs.umass.edu)

Related: TCP vs UDP and the handshake · What happens when you type a URL

L1 · CORE CORE CS

TCP vs UDP and the handshake ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked in nearly every CN round; the 'why not 2-way?' follow-up catches people out.

The concept

TCP is connection-oriented and reliable: ordered delivery, retransmission, flow control (receiver window) and congestion control (slow start, congestion avoidance, fast retransmit). UDP is connectionless with no guarantees but low overhead and latency.

Handshake: client SYN (seq x) → server SYN-ACK (seq y, ack x+1) → client ACK (ack y+1). Three steps are needed so both sides confirm the other received their initial sequence number; two would leave the server unsure the client saw its SYN, and stale duplicate SYNs could open ghost connections.

Closing uses FIN/ACK in four steps; TIME_WAIT keeps the socket around to absorb late packets.

Key points

- Use UDP for video calls, games, DNS, live streams; QUIC builds reliability on UDP
- Flow control protects the receiver; congestion control protects the network
- SYN flood attack exploits half-open connections (SYN cookies defend)

Common mistakes

× Mixing flow and congestion control

Interview questions

☐ Why a 3-way and not 2-way handshake?
☐ When would you choose UDP?
☐ Flow vs congestion control?

Learn from

Kurose & Ross (gaia.cs.umass.edu) · Gate Smashers (YouTube) (youtube.com)

Related: OSI and TCP/IP models · HTTP, HTTPS and TLS

L2 · INTERVIEW-READY CORE CS

HTTP, HTTPS and TLS ↑ Index

> **WHY IT MATTERS FOR YOU**
> Bridges CN and security. Your AppSec and cloud work depends on it.

The concept

HTTP is a stateless request/response protocol: methods (GET, POST, PUT, DELETE, PATCH), status codes (2xx ok, 3xx redirect, 4xx client error, 5xx server error), headers, cookies for state.

HTTPS is HTTP over TLS. In TLS 1.3 the client and server agree keys with an ephemeral Diffie-Hellman exchange, the server proves its identity with a certificate signed by a trusted CA, and then traffic is encrypted with a fast symmetric cipher (AES-GCM or ChaCha20).

HTTP/2 multiplexes streams over one TCP connection; HTTP/3 runs over QUIC (UDP) to avoid head-of-line blocking.

Key points

- Asymmetric crypto for key exchange/auth, symmetric for bulk data
- Security headers: HSTS, CSP, X-Frame-Options
- Idempotent methods: GET, PUT, DELETE

Common mistakes

× Saying HTTPS encrypts with the server's public key directly (old RSA key exchange, not TLS 1.3)

Interview questions

☐ How does HTTPS work?
☐ Difference between HTTP/1.1, 2 and 3?
☐ What are cookies and how are they secured?

Learn from

The Illustrated TLS 1.3 Connection `(tls13.xargs.org)` · MDN HTTP docs `(developer.mozilla.org)`

Related: TCP vs UDP and the handshake · What happens when you type a URL · Hashing, encryption and passwords

L2 · INTERVIEW-READY CORE CS

What happens when you type a URL ↑ Index

> **WHY IT MATTERS FOR YOU**
> The single most popular CN interview question. It lets you show breadth from DNS to rendering.

The concept

1) Browser checks its caches and parses the URL. 2) DNS resolution: browser → OS → resolver → root → TLD (.com) → authoritative server, giving an IP. 3) TCP handshake (or QUIC). 4) TLS handshake. 5) HTTP request sent; it may pass through a CDN, load balancer and reverse proxy. 6) The server builds a response (app server, cache, database). 7) The browser parses HTML, fetches CSS/JS/images, builds the DOM and CSSOM, lays out and paints.

Go deeper on whichever layer the interviewer prods: DNS record types (A, AAAA, CNAME, MX), TTL caching, ARP on the local network, keep-alive.

Key points

- DNS uses UDP port 53 (TCP for large responses)
- Mention a CDN and load balancer to show system awareness
- Security add-on: HSTS, certificate validation

Common mistakes

× Skipping DNS or TLS

× Going so deep on one step you never finish

Interview questions

☐ What happens when you type google.com and press Enter?

☐ How does DNS resolution work?

Learn from

alex/what-happens-when (GitHub) (github.com)

Related: OSI and TCP/IP models · TCP vs UDP and the handshake · HTTP, HTTPS and TLS · Caching and CDNs

L1 · CORE CORE CS

OOP: the four pillars ↑ Index

> **WHY IT MATTERS FOR YOU**
> Asked in every service-company interview and most product ones, often with 'show me in C++'.

The concept

Encapsulation: bundle data with methods and hide internals (private members, public interface). Abstraction: expose what an object does, not how (abstract classes, interfaces). Inheritance: a class reuses and extends another ('is-a'). Polymorphism: one interface, many forms; compile-time (overloading, templates) and run-time (virtual functions).

Prefer composition ('has-a') over deep inheritance trees.

Key points

- Overloading = same name, different parameters (compile time)
- Overriding = derived class redefines a virtual function (run time)
- Abstract class has at least one pure virtual function

Code

```
struct Shape { virtual double area() const = 0; virtual ~Shape() = default; };
struct Circle : Shape { double r; explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; } };
std::unique_ptr<Shape> s = std::make_unique<Circle>(2.0); // runtime polymorphism
```

Common mistakes

× Forgetting a virtual destructor in a base class

Interview questions

☐ Explain the four pillars with examples.
☐ Overloading vs overriding?
☐ Abstract class vs interface?

Learn from

learncpp.com: OOP chapters (learncpp.com) · GfG: OOP in C++ (geeksforgeeks.org)

Related: Virtual functions and the vtable · SOLID and design patterns · Low-level design (LLD)

L2 · INTERVIEW-READY CORE CS

Virtual functions and the vtable ↑ Index

> **WHY IT MATTERS FOR YOU**
> A C++-specific depth question that separates prepared candidates, and essential for HFT and binary exploitation.

The concept

Each class with virtual functions has a vtable: an array of function pointers to its versions of those functions. Each object stores a hidden vptr pointing to its class's vtable.

A virtual call loads the vptr, indexes the vtable and jumps: one extra indirection, and it usually blocks inlining (why low-latency C++ avoids virtual calls on hot paths, using templates/CRTP instead).

Constructors set the vptr, so calling a virtual function inside a constructor uses the current class's version, not the derived one.

Key points

- Virtual destructor needed for delete through a base pointer
- Object slicing: copying a derived object into a base by value
- Overwriting a vptr is a classic exploitation primitive

Common mistakes

× Thinking vtables are per object (they're per class)

Interview questions

☐ How does a virtual function call work?
☐ Why should base destructors be virtual?
☐ What is object slicing?

Learn from

learncpp: the virtual table (learncpp.com)

Related: OOP: the four pillars · Binary exploitation basics

L2 · INTERVIEW-READY CORE CS

SOLID and design patterns ↑ Index

> **WHY IT MATTERS FOR YOU**
> The vocabulary of every LLD round.

The concept

S: single responsibility. O: open for extension, closed for modification. L: Liskov substitution (a subclass must work wherever the base does). I: interface segregation (small, focused interfaces). D: dependency inversion (depend on abstractions, inject dependencies).

Patterns to know cold: Singleton (one instance; thread-safe via a function-local static in C++11), Factory (create objects without naming concrete classes), Strategy (swap algorithms at runtime), Observer (subscribers notified of events), plus Decorator and Adapter.

Key points

- Square-Rectangle is the classic Liskov violation
- Strategy = pass a pricing/parking-fee policy object
- Observer = notification systems

Common mistakes

× Naming patterns without a concrete use case

Interview questions

☐ Explain SOLID with examples.
☐ Implement a thread-safe singleton.
☐ Where would you use the strategy pattern?

Learn from

Refactoring.Guru design patterns (free) (refactoring.guru) · awesome-low-level-design (github.com)

Related: OOP: the four pillars · Low-level design (LLD)

L2 · INTERVIEW-READY Q&A

Operating systems: model answers ↑ Index

> **WHY IT MATTERS FOR YOU**
> OS is asked in nearly every technical round. These are the questions that come up most.

The concept

Read the question, answer it out loud from memory, then open the model answer and compare. Mark the ones you missed and re-do them in 3 days.

Questions and model answers

01. What is the difference between a process and a thread?

A process is a running program with its own address space, open files and a process control block. A thread is a unit of execution inside a process; threads of the same process share code, heap and files but each has its own stack, registers and program counter. Threads are cheaper to create and switch between because no new address space is needed and the TLB isn't flushed. The cost is safety: a bug or race in one thread can corrupt the whole process, so shared data needs synchronisation.

02. What happens during a context switch?

The kernel saves the running task's CPU state (registers, program counter, stack pointer) into its PCB or thread control block, picks the next task with the scheduler, restores that task's saved state and, if it belongs to another process, switches the page tables, which also invalidates TLB entries. Context switches are pure overhead, which is why too small a round-robin time quantum hurts throughput.

03. What are the four conditions for deadlock, and how do you prevent it?

Mutual exclusion, hold and wait, no preemption and circular wait; all four must hold. Prevention breaks one of them. In practice the easiest is circular wait: acquire locks in one fixed global order. Others: request all resources at once (breaks hold and wait), or allow preemption. Avoidance (Banker's algorithm) only grants a request if the system stays in a safe state. Detection builds a wait-for graph and recovers by killing or rolling back a process.

04. Mutex vs semaphore?

A mutex is a lock with an owner: one thread locks it and the same thread must unlock it, giving mutual exclusion for a critical section. A semaphore is a counter with wait (decrement, block at zero) and signal (increment). A counting semaphore lets up to N threads use N identical resources, like a pool of 5 database connections. A binary semaphore looks like a mutex but has no owner, so it's also used for signalling between threads, for example producer to consumer.

05. What is virtual memory and why is it used?

Each process sees a private, contiguous address space. The MMU translates virtual pages to physical frames through page tables, with the TLB caching recent translations. Benefits: isolation between processes, running programs larger than RAM by keeping only the working set in memory, simpler loading and linking, and sharing (shared libraries, copy-on-write after fork). The cost is translation overhead and page faults.

06. What is a page fault and what is thrashing?

A page fault occurs when a process touches a page that isn't in RAM. The OS traps, finds the page on disk, picks a victim frame with a replacement policy such as LRU, loads the page, updates the page table and restarts the instruction. Thrashing is when the combined working sets don't fit in memory, so the system spends most of its time paging instead of executing. Fixes: reduce the degree of multiprogramming or add memory.

07. Compare FCFS, SJF and round robin scheduling.

FCFS runs jobs in arrival order; simple but short jobs wait behind long ones (convoy effect). SJF picks the shortest burst and gives the minimum average waiting time, but needs burst estimates and can starve long jobs. Round robin gives each process a time quantum in turn: good response time for interactive systems; the quantum must be large relative to the context-switch cost. Real systems use multilevel feedback queues or Linux's CFS.

08. What is a race condition, and what is a critical section?

A race condition is when the result depends on the timing of threads accessing shared data, for example two threads doing count++ (load, add, store) and losing an update. The critical section is the code that touches the shared data. A correct solution needs mutual exclusion, progress and bounded waiting, implemented with locks, semaphores or atomic operations.

09. What are zombie and orphan processes?

A zombie has finished but its parent hasn't called wait() to read its exit status, so its process-table entry remains. An orphan's parent died first; it's re-parented to init or systemd, which reaps it. Many zombies can exhaust process IDs, so parents must wait() or handle SIGCHLD.

10. What do fork() and exec() do?

fork() creates a child that is a copy of the parent; modern kernels share pages copy-on-write until either side writes. It returns 0 in the child and the child's PID in the parent. exec() replaces the current process image with a new program, keeping the PID and open file descriptors. A shell runs a command with fork, then exec in the child, while the parent waits.

11. Internal vs external fragmentation?

Internal fragmentation is wasted space inside an allocated block, for example the unused part of the last page. External fragmentation is enough total free memory split into pieces too small to use, typical of variable-size allocation and segmentation. Paging removes external fragmentation at the cost of some internal fragmentation.

Learn from

OSTEP (free book) `(pages.cs.wisc.edu)`

Related: Processes and threads · Deadlock · Memory management and virtual memory · Synchronisation: mutex, semaphore, race conditions · CPU scheduling

L2 · INTERVIEW-READY Q&A

DBMS and SQL: model answers ↑ Index

> **WHY IT MATTERS FOR YOU**
> Every backend and fintech interview asks these; Razorpay-style companies go deeper on transactions.

The concept

Read the question, answer it out loud from memory, then open the model answer and compare. Mark the ones you missed and re-do them in 3 days.

Questions and model answers

01. Explain normalisation up to 3NF with an example.

Take Student(roll, name, dept, dept_head). 1NF: every value is atomic, no repeating groups. 2NF: no non-key column depends on part of a composite key. 3NF: no non-key column depends on another non-key column. dept_head depends on dept, not on roll, so it's a transitive dependency; split into Student(roll, name, dept) and Dept(dept, dept_head). This removes update anomalies: changing a head now touches one row.

02. What are the ACID properties?

Atomicity: a transaction happens fully or not at all, using undo logs. Consistency: constraints hold before and after. Isolation: concurrent transactions behave as if run one after another, to the chosen isolation level. Durability: once committed, data survives crashes, using write-ahead logging. Example: a bank transfer debits one account and credits another; atomicity stops money disappearing if the system crashes in between.

03. Which isolation level prevents dirty reads? Explain the anomalies.

A dirty read is reading another transaction's uncommitted data; a non-repeatable read is the same row returning different values within one transaction; a phantom read is a repeated range query returning new rows. Read Committed prevents dirty reads, Repeatable Read also prevents non-repeatable reads, and Serializable prevents phantoms. Higher isolation costs concurrency. PostgreSQL and MySQL implement these with MVCC snapshots.

04. How does an index speed up a query, and why B+ trees?

Without an index the database scans every row. A B+ tree index keeps keys sorted in wide, shallow nodes sized to disk pages, so finding a key among millions takes about 3 to 4 page reads, and range queries walk the linked leaf level. Hash indexes are faster for exact matches but can't do ranges. Indexes speed reads but slow inserts and updates and use space.

05. Clustered vs non-clustered index?

A clustered index determines the physical order of the rows; there's one per table, usually the primary key. A non-clustered (secondary) index is a separate structure storing the key plus a pointer or the primary key, so lookups may need a second step. A covering index includes all columns the query needs and avoids that step.

06. Primary key vs unique key vs foreign key?

A primary key uniquely identifies each row and can't be null; one per table. A unique key also enforces uniqueness but allows null (one or many nulls depending on the database) and a table can have several. A foreign key references another table's key and enforces referential integrity, for example orders.user_id must exist in users.id.

07. DELETE vs TRUNCATE vs DROP?

DELETE removes chosen rows with WHERE, is logged row by row, fires triggers and can be rolled back. TRUNCATE removes all rows quickly by deallocating pages, usually can't use WHERE and resets identity counters. DROP removes the table itself along with its structure, indexes and constraints.

08. Write SQL for the second-highest salary per department.

Use a window function: SELECT dept_id, name, salary FROM (SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r FROM employee e) t WHERE r = 2. DENSE_RANK handles ties so two people sharing the top salary don't push the second-highest to rank 3. Without window functions, use a correlated subquery counting distinct higher salaries.

09. What is the difference between WHERE and HAVING?

WHERE filters rows before grouping and can't use aggregates. HAVING filters groups after GROUP BY and can use aggregates, for example SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 5. The logical order is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

10. SQL vs NoSQL: when would you choose each?

SQL databases give a fixed schema, joins and strong ACID transactions: good for payments, orders, anything relational. NoSQL stores (document, key-value, wide-column, graph) give flexible schemas and easier horizontal scaling, often with eventual consistency: good for huge write volumes, caching, feeds and logs. Many systems use both, for example PostgreSQL for orders and Redis for sessions.

11. How do you prevent SQL injection?

Never build queries by concatenating user input. Use parameterised queries or prepared statements, where the query structure and the data are sent separately, so input can't change the query. Add least-privilege database accounts and input validation as extra layers.

Learn from

CMU 15-445 (free lectures) `(15445.courses.cs.cmu.edu)` · Use The Index, Luke `(use-the-index-luke.com)`

Related: Transactions, ACID and isolation · Normalisation and keys · Indexing and B+ trees · SQL for interviews

L2 · INTERVIEW-READY Q&A

Computer networks: model answers ↑ Index

> **WHY IT MATTERS FOR YOU**
> CN questions are short but frequent, and they connect directly to your security work.

The concept

Read the question, answer it out loud from memory, then open the model answer and compare. Mark the ones you missed and re-do them in 3 days.

Questions and model answers

01. What happens when you type google.com and press Enter?

The browser checks its caches, then resolves the name through DNS: OS cache, then a recursive resolver that asks the root, the .com TLD server and Google's authoritative server, returning an IP. The browser opens a TCP connection (three-way handshake) or QUIC, then a TLS handshake that checks Google's certificate and agrees keys. It sends an HTTP GET; the request may pass a CDN and load balancer to an application server. The response HTML is parsed, CSS, JS and images are fetched, and the page is laid out and painted.

02. Explain the TCP three-way handshake. Why not two?

Client sends SYN with sequence number x; server replies SYN-ACK with its own sequence y and ack x+1; client sends ACK y+1. Three messages are needed so both sides know the other received their initial sequence number. With two, the server couldn't tell whether the client got its SYN, and an old duplicate SYN could open a ghost connection.

03. TCP vs UDP, and when would you use UDP?

TCP is connection-oriented and reliable: ordered delivery, acknowledgements, retransmission, flow and congestion control. UDP sends independent datagrams with no guarantees and very little overhead. Use UDP when latency matters more than perfect delivery: video calls, online games, live streaming, DNS lookups. QUIC (HTTP/3) adds reliability on top of UDP to avoid TCP's head-of-line blocking.

04. Flow control vs congestion control?

Flow control protects the receiver: the receiver advertises a window of how much it can buffer, and the sender never exceeds it. Congestion control protects the network: the sender keeps a congestion window that grows with slow start and additive increase, and shrinks sharply on packet loss (multiplicative decrease).

05. Explain the OSI model and where common devices sit.

Physical (bits on the wire), data link (frames, MAC addresses, switches), network (IP packets, routing, routers), transport (TCP/UDP, ports), session, presentation and application (HTTP, DNS). TCP/IP merges them into link, internet, transport and application. Each layer adds its own header: data, then segment, packet and frame.

06. How does HTTPS work?

HTTPS is HTTP over TLS. In TLS 1.3 the client and server exchange ephemeral Diffie-Hellman key shares to derive a shared secret; the server proves its identity with a certificate signed by a trusted certificate authority, which the client verifies. After the handshake, traffic is encrypted and integrity-protected with a fast symmetric cipher such as AES-GCM. Asymmetric crypto is used only to authenticate and agree keys because it's slow.

07. What is DNS and which record types matter?

DNS maps names to addresses through a hierarchy of servers with caching controlled by TTLs. A maps to IPv4, AAAA to IPv6, CNAME aliases one name to another, MX gives mail servers, NS names authoritative servers, TXT holds text such as SPF records. It mostly uses UDP port 53, falling back to TCP for large responses.

08. What is subnetting? How many hosts in a /26?

Subnetting splits a network by borrowing host bits for the network part. A /26 leaves 32 − 26 = 6 host bits: $2^6 = 64$ addresses, minus the network and broadcast addresses, so 62 usable hosts. Subnetting limits broadcast domains and lets you apply security rules per subnet, like public and private subnets in an AWS VPC.

09. What is ARP, and what is ARP spoofing?

ARP finds the MAC address for an IP on the local network: a host broadcasts 'who has 192.168.1.1?' and the owner replies with its MAC. ARP has no authentication, so an attacker can send forged replies and become the man in the middle. Defences include dynamic ARP inspection on switches and encrypting traffic with TLS.

10. HTTP/1.1 vs HTTP/2 vs HTTP/3?

HTTP/1.1 sends requests one after another per connection (browsers open several connections). HTTP/2 multiplexes many streams over one TCP connection with header compression, but a lost TCP packet still blocks all streams. HTTP/3 runs over QUIC on UDP, so streams are independent and connection setup is faster.

Learn from

Kurose and Ross `(gaia.cs.umass.edu)` · The Illustrated TLS 1.3 Connection `(tls13.xargs.org)`

Related: OSI and TCP/IP models · TCP vs UDP and the handshake · HTTP, HTTPS and TLS · What happens when you type a URL

L2 · INTERVIEW-READY Q&A

OOP and C++: model answers ↑ Index

> **WHY IT MATTERS FOR YOU**
> Every service-company round and most product rounds ask OOP; C++ depth is a plus for HFT.

The concept

Read the question, answer it out loud from memory, then open the model answer and compare. Mark the ones you missed and re-do them in 3 days.

Questions and model answers

01. Explain the four pillars of OOP with examples.

Encapsulation: keep data private and expose methods, like a BankAccount whose balance changes only through deposit and withdraw. Abstraction: show what an object does, not how, like an abstract Shape with area(). Inheritance: a Circle is a Shape and reuses its interface. Polymorphism: calling area() on a Shape reference runs Circle's or Square's version at run time through virtual functions; overloading and templates give compile-time polymorphism.

02. Overloading vs overriding?

Overloading is the same function name with different parameter lists in the same scope, resolved at compile time. Overriding is a derived class redefining a base class's virtual function with the same signature, resolved at run time through the vtable. In C++, mark overrides with the override keyword so the compiler catches signature mistakes.

03. How does a virtual function call work in C++?

Each class with virtual functions has a vtable, an array of pointers to its versions of those functions. Each object holds a hidden vptr to its class's vtable, set by the constructor. A virtual call loads the vptr, indexes the vtable and jumps: one extra indirection that usually prevents inlining.

04. Why should a base class destructor be virtual?

If you delete a derived object through a base pointer and the destructor isn't virtual, only the base destructor runs; the derived part's resources leak and the behaviour is undefined. Any class meant to be used polymorphically should have a virtual destructor.

05. Abstract class vs interface?

In C++ an abstract class has at least one pure virtual function (= 0) and can't be instantiated; it can still have data and implemented methods. An interface is the special case with only pure virtual functions and no data. Java and C# have a separate interface keyword; C++ uses abstract classes for both.

06. What is RAII?

Resource Acquisition Is Initialisation: tie a resource's lifetime to an object's lifetime. The constructor acquires (memory, file, lock) and the destructor releases, so cleanup happens automatically even when an exception is thrown. unique_ptr, lock_guard and ifstream are RAII types; it's why modern C++ rarely needs raw new and delete.

07. unique_ptr vs shared_ptr?

unique_ptr has one owner, can be moved but not copied, and has no overhead over a raw pointer. shared_ptr allows many owners through a reference count and frees the object when the count reaches zero; it costs an atomic counter and can leak through cycles, which weak_ptr breaks. Default to unique_ptr.

08. Explain SOLID briefly.

Single responsibility: one reason to change. Open/closed: extend behaviour without editing existing code, through interfaces. Liskov substitution: a subclass must work anywhere its base does (a Square that breaks Rectangle's setters violates it). Interface segregation: small focused interfaces. Dependency inversion: depend on abstractions and inject concrete implementations.

09. Shallow copy vs deep copy?

A shallow copy copies member values, so two objects end up pointing at the same heap memory, leading to double frees. A deep copy allocates new memory and copies the pointed-to data. If a class owns a raw resource, follow the rule of three/five (destructor, copy and move operations), or use RAII members like vector so the defaults are correct.

10. What is a static member, and what is const correctness?

A static data member belongs to the class, not to any object, so all instances share it; a static member function has no this pointer. Const correctness means marking anything that shouldn't change as const: const references for parameters you only read, and const member functions that promise not to modify the object.

Learn from

learncpp.com `(learncpp.com)` · Refactoring.Guru `(refactoring.guru)`

Related: OOP: the four pillars · Virtual functions and the vtable · SOLID and design patterns

L2 · INTERVIEW-READY Q&A

Security and cloud: model answers ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your edge as a cybersecurity student: SDE interviewers remember a strong security answer.

The concept

Read the question, answer it out loud from memory, then open the model answer and compare. Mark the ones you missed and re-do them in 3 days.

Questions and model answers

01. Authentication vs authorisation?

Authentication verifies who you are (password, MFA, certificate). Authorisation decides what you may do once identified (roles, policies, ownership checks). Broken access control, the top OWASP risk, is an authorisation failure: the user is logged in but the app doesn't check they own the object they request.

02. Hashing vs encryption, and how should passwords be stored?

Hashing is one-way and fixed-length, used for integrity. Encryption is reversible with a key. Passwords should be hashed with a slow, salted algorithm such as Argon2id or bcrypt: the salt defeats precomputed tables and the slowness makes brute force expensive. Never encrypt passwords or use a fast hash like MD5 or SHA-256 alone.

03. What is XSS and how do you prevent it?

Cross-site scripting runs attacker-controlled JavaScript in another user's browser on your site, through reflected input, stored content or client-side DOM writes. Prevent it with context-aware output encoding, frameworks that escape by default, a Content-Security-Policy, and HttpOnly cookies so stolen scripts can't read session tokens.

04. What is CSRF?

Cross-site request forgery makes a logged-in user's browser send an unwanted request, such as changing their email, from another site, because the browser attaches cookies automatically. Defences: SameSite cookies, anti-CSRF tokens tied to the session, and re-authentication for sensitive actions.

05. What is SSRF and why is it dangerous in the cloud?

Server-side request forgery makes your server fetch a URL the attacker controls. They point it at internal services or the cloud metadata endpoint at 169.254.169.254 to steal temporary IAM credentials. Defend with allow-lists, blocking private address ranges after DNS resolution, IMDSv2 on AWS, and least-privilege instance roles.

06. How does AWS IAM decide whether a request is allowed?

Everything starts denied. AWS evaluates all applicable policies: identity policies, resource policies, permission boundaries and organisation SCPs. An explicit Deny anywhere wins; otherwise an Allow is needed and nothing may cap it. That's why TenantWall uses explicit denies to enforce tenant isolation even if someone adds a broad allow later.

07. What is ABAC and why use it for multi-tenant isolation?

Attribute-based access control grants access when attributes match, for example a principal's tenant tag equals a resource's tenant tag. One policy covers every tenant, so onboarding a new tenant needs no new policy, unlike role-per-tenant RBAC. The tags themselves must be protected so users can't re-tag themselves into another tenant.

08. How would you detect unauthorised access attempts in AWS?

Enable CloudTrail, including S3 data events for sensitive buckets, and send it to CloudWatch Logs. Create a metric filter matching AccessDenied and UnauthorizedOperation error codes, put an alarm on the metric, and notify through SNS. GuardDuty adds managed threat detection. This is the monitoring layer in TenantWall.

09. Explain a buffer overflow and the main mitigations.

Writing past the end of a stack buffer can overwrite the saved return address and redirect execution. Stack canaries detect the overwrite before return, NX/DEP stops executing injected code on the stack, and ASLR randomises addresses so the attacker must first leak one. Attackers respond with return-oriented programming and info leaks, so the real fix is bounds-checked code.

10. What is threat modelling?

A structured way to find design-level risks before attackers do: draw the data flow and trust boundaries, then for each element ask STRIDE (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege), and record a mitigation for each threat. It turns 'is it secure?' into a checklist you can defend in an interview.

Learn from

OWASP Cheat Sheet Series `(cheatsheetseries.owasp.org)` · AWS IAM policy evaluation `(docs.aws.amazon.com)`

Related: OWASP Top 10 · SQL injection · Cross-site scripting (XSS) and CSRF · AWS IAM and least privilege · Hashing, encryption and passwords

L1 · CORE DESIGN

Low-level design (LLD) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Freshers at product companies get LLD more often than HLD. Flipkart, Swiggy, Atlassian and others run a machine-coding or LLD round.

The concept

Process: clarify requirements and scope (5 min) → list entities (nouns) and behaviours (verbs) → define classes, fields and relationships → key interfaces → walk through one flow → discuss extensions, concurrency and patterns used.

Practise: parking lot, library management, Splitwise, elevator, snake and ladder, BookMyShow, rate limiter, LRU cache, vending machine.

Key points

- Draw a class diagram first
- Use enums for states and types
- Mention thread safety where shared state exists
- Machine-coding rounds want runnable, clean code in 60–90 min

Common mistakes

× Starting to code before agreeing on requirements
× God classes that do everything

Interview questions

☐ Design a parking lot.
☐ Design Splitwise.
☐ Design an elevator system.

Learn from

awesome-low-level-design (github.com) · Refactoring.Guru (refactoring.guru)

Related: SOLID and design patterns · OOP: the four pillars · High-level system design (HLD)

L2 · INTERVIEW-READY DESIGN

High-level system design (HLD) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Big Tech asks light HLD even for new grads; a clear design discussion lifts you above candidates who only grind DSA. Your AWS SAA prep covers half of it.

The concept

Framework: requirements (functional + non-functional: scale, latency, availability) → back-of-envelope numbers (QPS, storage) → API → data model → high-level diagram → deep dive on the bottleneck → trade-offs.

Building blocks: load balancer, stateless app servers, cache (Redis), CDN, SQL vs NoSQL, replication, sharding, message queues (SQS/Kafka), object storage (S3), rate limiter, search index.

Starter designs: URL shortener, pastebin, rate limiter, chat app, news feed, notification service.

Key points

- 1 day ≈ 10^5 seconds; 1M requests/day ≈ 12 QPS
- Read-heavy → cache + replicas; write-heavy → sharding + queues
- State the trade-off you chose and why

Common mistakes

- × Jumping to microservices for a simple system
- × No numbers

Interview questions

- ☐ Design a URL shortener.
- ☐ How would you scale it to 100× traffic?

Learn from

System Design Primer (github.com) · roadmap.sh system design (roadmap.sh) · ByteByteGo blog (free posts) (blog.bytebytego.com)

Related: Caching and CDNs · CAP theorem and consistency · Load balancing, replication and sharding · Low-level design (LLD)

L2 · INTERVIEW-READY DESIGN

Caching and CDNs ↑ Index

> **WHY IT MATTERS FOR YOU**
> The first scaling answer in any design question, and it links to LRU (a DSA favourite).

The concept

A cache keeps hot data in fast storage. Patterns: cache-aside (app reads cache, falls back to DB and fills cache), write-through (write cache and DB together), write-back (write cache, flush later).

Eviction: LRU, LFU, TTL. Invalidation is the hard part: set TTLs, delete on write, version keys.

A CDN caches static (and some dynamic) content at edge locations close to users (CloudFront on AWS).

Key points

- Cache stampede: many misses at once; use locking or early refresh
- Hit ratio is the metric
- LRU = hash map + doubly linked list

Common mistakes

× Caching data that must be strongly consistent without a plan

Interview questions

☐ How would you add caching to this system?

☐ How do you keep the cache consistent?

Learn from

System Design Primer: cache (github.com)

Related: High-level system design (HLD) · Linked lists

L2 · INTERVIEW-READY DESIGN

Load balancing, replication and sharding ↑ Index

> **WHY IT MATTERS FOR YOU**
> Core scaling vocabulary for HLD rounds and AWS SAA.

The concept

A load balancer spreads requests across servers (round robin, least connections, consistent hashing) and removes unhealthy ones via health checks. L4 balances by IP/port, L7 by HTTP content.

Replication copies data to more nodes for read scaling and availability (leader-follower; replication lag means followers can be stale). Sharding splits data across nodes by a key for write scaling; choose a key that spreads load evenly and avoids cross-shard queries.

Consistent hashing keeps most keys in place when nodes join or leave.

Key points

- AWS: ALB (L7), NLB (L4), Auto Scaling groups, RDS read replicas, DynamoDB partitions
- Hot shard problem: celebrity users

Common mistakes

× Sharding before trying caching and replicas

Interview questions

☐ How would you shard a users table?

☐ What is consistent hashing?

Learn from

System Design Primer (github.com)

Related: High-level system design (HLD) · CAP theorem and consistency · AWS core services and the SAA-C03 exam

L3 · ADVANCED DESIGN

CAP theorem and consistency ↑ Index

> **WHY IT MATTERS FOR YOU**
> An L3 depth question that shows you understand distributed trade-offs.

The concept

During a network partition, a distributed store must choose between consistency (every read sees the latest write) and availability (every request gets a response). PACELC adds: Else, when there's no partition, choose between latency and consistency.

Consistency models range from strong (linearisable) to eventual. Banking ledgers lean to consistency; social feeds lean to availability.

Key points

- Quorums: R + W > N gives read-your-writes
- Idempotency keys make retries safe (payments)

Common mistakes

× Saying a system can 'pick two' outside partitions

Interview questions

☐ Explain CAP with an example.

☐ How do you make a payment API safe to retry?

Learn from

Designing Data-Intensive Applications (book) (dataintensive.net)

Related: High-level system design (HLD) · Transactions, ACID and isolation

L1 · CORE SECURITY

OWASP Top 10 ↑ Index

> **WHY IT MATTERS FOR YOU**
> The shared language of AppSec. Even SDE interviewers are impressed when you name the risk in your own project.

The concept

The OWASP Top 10 (2021) lists the most critical web risks: A01 Broken Access Control, A02 Cryptographic Failures, A03 Injection (incl. XSS), A04 Insecure Design, A05 Security Misconfiguration, A06 Vulnerable and Outdated Components, A07 Identification and Authentication Failures, A08 Software and Data Integrity Failures, A09 Security Logging and Monitoring Failures, A10 Server-Side Request Forgery.

Broken access control is #1: the app checks that you are logged in but not that you may access this object.

Key points

- Authentication = who you are; authorisation = what you may do
- Defence in depth: validate input, encode output, least privilege, logging
- A09 is exactly what TenantWall's CloudTrail + CloudWatch alarm addresses

Common mistakes

× Memorising the list without one exploit + one fix for each

Interview questions

☐ Name three OWASP risks and how you'd prevent them.

☐ Authentication vs authorisation?

Practise

PortSwigger Web Security Academy `(portswigger.net)`

Learn from

OWASP Top 10 `(owasp.org)` · OWASP Cheat Sheet Series `(cheatsheetseries.owasp.org)`

Related: SQL injection · Cross-site scripting (XSS) and CSRF · Broken access control and IDOR · Server-side request forgery (SSRF)

L1 · CORE SECURITY

SQL injection ↑ Index

> **WHY IT MATTERS FOR YOU**
> The classic vulnerability; first PortSwigger topic and a guaranteed security interview question.

The concept

Happens when user input is concatenated into a SQL query, so the input changes the query's structure. ' OR '1'='1 turns a login check into 'always true'; UNION SELECT reads other tables; blind SQLi infers data from true/false or timing differences.

Fix: parameterised queries / prepared statements, which send the query and the data separately so data can never become code. Add least-privilege DB accounts and input validation as extra layers. Escaping by hand is fragile.

Key points

- ORMs are safe unless you build raw queries with strings
- Second-order SQLi: stored input used later in a query

Example

```
// vulnerable
q = "SELECT * FROM users WHERE name = '" + name + "'";
// safe (prepared statement)
stmt = db.prepare("SELECT * FROM users WHERE name = ?"); stmt.bind(1, name);
```

Common mistakes

× Thinking input filtering alone fixes it

Interview questions

☐ What is SQL injection and how do you prevent it?
☐ Why do prepared statements work?

Practise

PortSwigger: SQL injection labs (portswigger.net)

Learn from

PortSwigger: SQL injection (portswigger.net) · OWASP SQLi prevention cheat sheet (cheatsheetseries.owasp.org)

Related: OWASP Top 10 · SQL for interviews

L1 · CORE SECURITY

Cross-site scripting (XSS) and CSRF ↑ Index

> **WHY IT MATTERS FOR YOU**
> The two most common client-side web bugs; asked in AppSec and full-stack interviews.

The concept

XSS: attacker-controlled script runs in another user's browser on your site. Reflected (from the request), stored (saved in the database), DOM-based (client JS writes untrusted data into the page). Impact: steal session data, act as the user.

Fix XSS: context-aware output encoding (HTML, attribute, JS, URL), frameworks that escape by default, a Content-Security-Policy, HttpOnly cookies.

CSRF: a malicious site makes the victim's browser send an authenticated request (e.g. change email). Fix: SameSite cookies, anti-CSRF tokens, re-authentication for sensitive actions.

Key points

- innerHTML with user data → DOM XSS
- SameSite=Lax is the modern default in browsers
- CORS is not a CSRF defence

Common mistakes

× Confusing XSS (runs script) with CSRF (forges a request)

Interview questions

☐ Difference between XSS and CSRF?
☐ How does CSP help?

Practise

PortSwigger: XSS labs (portswigger.net) · PortSwigger: CSRF labs (portswigger.net)

Learn from

PortSwigger: XSS (portswigger.net) · PortSwigger: CSRF (portswigger.net)

Related: OWASP Top 10 · HTTP, HTTPS and TLS

L2 · INTERVIEW-READY SECURITY

Broken access control and IDOR ↑ Index

> **WHY IT MATTERS FOR YOU**
> OWASP #1 and the most common bug-bounty finding. Directly related to TenantWall's tenant isolation.

The concept

IDOR (insecure direct object reference): the app uses an ID from the request (/invoice?id=1043) without checking the object belongs to the user. Change the ID, read someone else's data.

Other forms: missing function-level checks (normal user calls /admin API), privilege escalation via a role parameter, path traversal.

Fix: enforce authorisation on the server for every request and object (deny by default), centralise checks, use unguessable IDs only as extra defence, test with two accounts.

Key points

- Multi-tenant SaaS: every query filtered by tenant_id from the session, never from the request
- ABAC in TenantWall enforces this at the AWS layer with tags

Common mistakes

× Relying on hidden UI elements as access control

Interview questions

☐ What is IDOR and how would you test for it?

☐ How do you enforce tenant isolation?

Practise

PortSwigger: access control labs (portswigger.net)

Learn from

PortSwigger: access control (portswigger.net)

Related: OWASP Top 10 · ABAC: attribute-based access control · AWS IAM and least privilege

L2 · INTERVIEW-READY SECURITY

Server-side request forgery (SSRF) ↑ Index

> **WHY IT MATTERS FOR YOU**
> OWASP A10 and the bug behind major cloud breaches; ties web security to AWS.

The concept

The server fetches a URL the attacker controls (image preview, webhook, PDF generator). The attacker points it at internal addresses: localhost admin panels, internal services, or the cloud metadata endpoint (169.254.169.254) to steal temporary IAM credentials.

Fix: allow-lists of destinations, block private and link-local ranges after DNS resolution, disable redirects, and on AWS require IMDSv2 (session tokens) and give instances least-privilege roles.

Key points

- The 2019 Capital One breach involved SSRF + over-privileged IAM role
- flaws.cloud level 5 shows the metadata attack

Common mistakes

× Blocking by string matching (bypassed with decimal IPs, DNS rebinding)

Interview questions

☐ What is SSRF and why is it dangerous in the cloud?

Practise

PortSwigger: SSRF labs (portswigger.net) · flaws.cloud (flaws.cloud)

Learn from

PortSwigger: SSRF (portswigger.net)

Related: OWASP Top 10 · AWS IAM and least privilege · AWS core services and the SAA-C03 exam

L1 · CORE SECURITY

Hashing, encryption and passwords ↑ Index

> **WHY IT MATTERS FOR YOU**
> A common short question in security and backend interviews.

The concept

Hashing is one-way and fixed-length (SHA-256): used for integrity and fingerprints. Encryption is reversible with a key: symmetric (AES, one shared key, fast) or asymmetric (RSA/ECC, public/private key pair, used for key exchange and signatures).

Passwords: never encrypt, never plain-hash. Use a slow, salted password hash (Argon2id, bcrypt, scrypt). The salt stops precomputed rainbow tables; slowness stops brute force.

Digital signatures: hash the message, sign with the private key; anyone verifies with the public key.

Key points

- Encoding (Base64) is not encryption
- MAC/HMAC = integrity + authenticity with a shared key
- AES-GCM gives encryption + integrity

Common mistakes

× Saying 'we hash with MD5' for passwords

Interview questions

☐ Hashing vs encryption?
☐ How should passwords be stored?
☐ Symmetric vs asymmetric?

Learn from

OWASP Password Storage cheat sheet (cheatsheetseries.owasp.org) · CryptoHack (free) (cryptohack.org)

Related: HTTP, HTTPS and TLS · OWASP Top 10

L2 · INTERVIEW-READY SECURITY

Threat modelling (STRIDE) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Lets you add 'I threat-modelled it' to any project answer. Security-engineer interviews ask you to do it live.

The concept

Draw the system's data flow: users, processes, data stores, trust boundaries. For each element, ask STRIDE: Spoofing (identity), Tampering (data), Repudiation (no audit trail), Information disclosure, Denial of service, Elevation of privilege.

For each threat, record a mitigation and its priority. Four questions: what are we building, what can go wrong, what will we do about it, did we do a good job?

Key points

- Trust boundaries are where most bugs live
- Repudiation → logging (CloudTrail)
- Elevation of privilege → IAM least privilege

Common mistakes

× A list of threats with no mitigations

Interview questions

☐ Threat-model a login page.

☐ Threat-model your project.

Learn from

OWASP Threat Modeling (owasp.org) · Microsoft STRIDE (learn.microsoft.com)

Related: OWASP Top 10 · TenantWall (your research project)

L3 · ADVANCED SECURITY

Binary exploitation basics ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your L3 security rung and the binary taster from the roadmap. It makes your C/C++ and OS memory knowledge concrete, which HFT and systems interviewers value.

The concept

Stack layout: local buffers sit below the saved frame pointer and return address. Writing past a buffer (gets, strcpy without bounds) overwrites the return address and redirects execution.

Defences and bypasses: stack canaries (detect overwrite), NX/DEP (stack not executable → return-oriented programming chains existing code 'gadgets'), ASLR (randomised addresses → need an info leak), PIE, RELRO.

Other bug classes: format strings (printf(user)), use-after-free and double free on the heap, integer overflows. Tools: GDB with pwndbg, pwntools, Ghidra for reversing, checksec.

Key points

- Start with pwn.college's program-security modules
- Write a short write-up for every challenge you solve
- Secure C++: bounds-checked containers, avoid raw C strings

Common mistakes

× Jumping to heap exploitation before stack basics

Interview questions

☐ What is a buffer overflow and how do canaries and ASLR stop it?
☐ What is ROP?

Practise

pwn.college (pwn.college) · picoCTF (picoctf.org) · ROP Emporium (ropemporium.com)

Learn from

pwn.college (pwn.college) · Nightmare (free course) (guyinatuxedo.github.io)

Related: Memory management and virtual memory · Virtual functions and the vtable · CTFs and security labs

L2 · INTERVIEW-READY SECURITY

CTFs and security labs ↑ Index

> **WHY IT MATTERS FOR YOU**
> A proven hiring channel for security roles (see the KPMG story) and great resume proof.

The concept

Jeopardy CTFs have categories: web, pwn (binary exploitation), reverse engineering, crypto, forensics, misc. Each solved challenge gives a flag.

Path: OverTheWire Bandit (Linux) → picoCTF (all categories, beginner) → PortSwigger labs (web) → pwn.college (binary) → live CTFs on CTFtime with a team.

Write up each solve on a blog or GitHub: problem, approach, the key insight, the fix. Write-ups are what recruiters can read.

Key points

- Join or form a college team; India has many college CTFs
- Top-25% finish in a mid-rated CTF is a solid resume line

Common mistakes

× Only reading write-ups without solving

Interview questions

☐ Tell me about a CTF challenge you solved.

Practise

OverTheWire Bandit (overthewire.org) · picoCTF (picoctf.org) · CTFtime (ctftime.org) · TryHackMe (tryhackme.com)

Learn from

CTF101 (ctf101.org)

Related: Binary exploitation basics · Bug bounty and responsible disclosure

L4 · ELITE SECURITY

Bug bounty and responsible disclosure

↑ Index

> **WHY IT MATTERS FOR YOU**
> An accepted report is proof no college name can match. It's your L4 security goal.

The concept

Bug bounty programmes pay or credit researchers for reporting vulnerabilities within a defined scope. Read the scope and rules first; testing out of scope is illegal.

Start with recon on wide-scope programmes, focus on one bug class you know well (IDOR, access control, XSS), and read disclosed reports on HackerOne Hacktivity to learn what gets accepted.

Responsible disclosure: report privately, give details and impact, don't access more data than needed, wait for a fix before publishing.

Key points

- Quality report: steps to reproduce, impact, fix suggestion
- Indian govt sites: report to CERT-In
- Duplicates are normal at first

Common mistakes

× Running automated scanners against programmes that forbid them

Interview questions

☐ Walk me through a vulnerability you found.

Practise

HackerOne Hacktivity (hackerone.com) · Bugcrowd (bugcrowd.com)

Learn from

PortSwigger Academy (portswigger.net) · HackerOne Hacker101 (free) (hacker101.com)

Related: Broken access control and IDOR · CTFs and security labs

L1 · CORE CLOUD

AWS IAM and least privilege ↑ Index

> **WHY IT MATTERS FOR YOU**
> Week 1 of your AWS track and the heart of TenantWall. The most important AWS service for security roles.

The concept

Principals (users, roles) get permissions from JSON policies: Effect (Allow/Deny), Action, Resource, Condition. Roles give temporary credentials and are preferred over long-lived user keys.

Evaluation logic: everything starts as an implicit deny; an Allow grants access; an explicit Deny anywhere always wins (over any Allow). Permission boundaries and SCPs cap what can be allowed.

Tools from your professor's list: the IAM Policy Generator builds policy JSON; IAM Access Analyzer finds resources shared outside your account and validates policies; it can generate least-privilege policies from CloudTrail activity.

Key points

- Explicit deny > allow > implicit deny
- Never use the root account day to day; enable MFA
- Use roles for EC2/Lambda, not access keys

Example

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Deny",
    "Action": "s3:*",
    "Resource": "*",
    "Condition": {"Bool": {"aws:SecureTransport": "false"}}
  }]
}
```

Common mistakes

× Policies with "Action": "*" and "Resource": "*"

Interview questions

☐ Explain IAM policy evaluation.
☐ Role vs user?
☐ How do you achieve least privilege?

Learn from

AWS docs: policy evaluation logic (docs.aws.amazon.com) · AWS docs: IAM Access Analyzer (docs.aws.amazon.com) · flaws2.cloud (flaws2.cloud)

Related: ABAC: attribute-based access control · TenantWall (your research project) · CloudTrail and CloudWatch alarms

L3 · ADVANCED CLOUD

ABAC: attribute-based access control ↑ Index

> **WHY IT MATTERS FOR YOU**
> The core idea of your TenantWall paper.

The concept

RBAC grants by role; ABAC grants by matching attributes. In AWS, attributes are tags on principals (aws:PrincipalTag/tenant) and on resources (aws:ResourceTag/tenant). One policy can say 'allow only when the principal's tenant tag equals the resource's tenant tag', so adding a tenant needs no new policy.

For S3, use object or bucket tags, or prefix-based conditions (s3:prefix with ${aws:PrincipalTag/tenant}). Protect the tags themselves: deny changing tenant tags (aws:TagKeys, aws:RequestTag) so a user can't re-tag into another tenant.

Key points

- Scales better than one role per tenant
- Pair with an explicit deny on tag modification
- Test isolation with two tenant users (should get AccessDenied)

Example

```
"Condition": {
  "StringEquals": {
    "aws:ResourceTag/tenant": "${aws:PrincipalTag/tenant}"
  }
}
```

Common mistakes

× Letting principals edit their own tags

Interview questions

☐ RBAC vs ABAC?

☐ How does TenantWall stop tenant A reading tenant B's data?

Learn from

AWS docs: What is ABAC? (docs.aws.amazon.com) · AWS docs: ABAC tutorial (docs.aws.amazon.com)

Related: AWS IAM and least privilege · TenantWall (your research project) · Broken access control and IDOR

L2 · INTERVIEW-READY CLOUD

CloudTrail and CloudWatch alarms ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your professor asked for exactly this in TenantWall: log every access attempt and alert on denied ones.

The concept

CloudTrail records API calls (who, what, when, from where). Management events are on by default; S3 object-level reads and writes need data events enabled on the trail. Send the trail to CloudWatch Logs.

Create a metric filter on the log group with the pattern { ($.errorCode = "*UnauthorizedOperation") || ($.errorCode = "AccessDenied*") }, which turns matching events into a metric. Put a CloudWatch alarm on it (e.g. ≥ 1 in 5 minutes) whose action publishes to an SNS topic with an email subscription.

Test: as tenant A, try to read tenant B's object; you should get AccessDenied, see the event in CloudTrail, the metric tick up and an email arrive.

Key points

- Confirm the SNS email subscription or nothing arrives
- Data events cost money; limit them to TenantWall buckets
- This maps to OWASP A09 (logging and monitoring failures)

Common mistakes

× Expecting S3 GetObject in CloudTrail without data events

Interview questions

☐ How would you detect unauthorised access attempts in AWS?

Learn from

AWS docs: CloudTrail data events (docs.aws.amazon.com) · AWS docs: CloudWatch alarms for CloudTrail (docs.aws.amazon.com)

Related: TenantWall (your research project) · AWS IAM and least privilege

L3 · ADVANCED CLOUD

TenantWall (your research project) ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your strongest resume line: real cloud security research with a professor. It deserves a polished write-up, a diagram and a 2-minute interview story.

The concept

What: multi-tenant isolation on EC2 and S3 using ABAC (tag-matched IAM policies), so tenants can't reach each other's resources.

Your professor's additions: a CloudTrail trail with S3 data events; a CloudWatch metric filter on AccessDenied/UnauthorizedOperation with an alarm and SNS email; policies built with the Policy Generator and checked with IAM Access Analyzer; a flowchart and an AWS-icon architecture diagram; explicit deny statements (confirm the exact meaning with your professor).

Plan: deadline 21 Mar 2027. Build in Dec–Jan, run isolation tests and collect evidence (screenshots, alarm emails), write in Feb–Mar.

Key points

- Interview story: problem → design (ABAC) → how you proved isolation → what the alarm caught
- Diagram in diagrams.net with the AWS icon set
- Put the policies and test scripts on GitHub (no keys!)

Common mistakes

- × Committing AWS keys to GitHub
- × Leaving resources running and getting billed

Interview questions

- ☐ Tell me about your project.
- ☐ How did you verify isolation?
- ☐ What would you change at 1,000 tenants?

Learn from

diagrams.net (AWS icons) `(app.diagrams.net)` · AWS docs: ABAC `(docs.aws.amazon.com)`

Related: ABAC: attribute-based access control · CloudTrail and CloudWatch alarms · AWS IAM and least privilege · Threat modelling (STRIDE)

L2 · INTERVIEW-READY CLOUD

AWS core services and the SAA-C03 exam ↑ Index

> **WHY IT MATTERS FOR YOU**
> 30 minutes a day from 16 Nov, exam in the week of 19 Apr 2027. Cloud skills plus a cert lift both SDE and security profiles.

The concept

Compute: EC2 (instances, AMIs, security groups), Auto Scaling, Lambda. Storage: S3 (classes, versioning, lifecycle, encryption, Block Public Access), EBS, EFS. Databases: RDS (Multi-AZ for HA, read replicas for scale), DynamoDB, ElastiCache. Networking: VPC, public/private subnets, route tables, internet and NAT gateways, ALB/NLB, Route 53, CloudFront. Integration: SQS, SNS. Security: IAM, KMS, CloudTrail, GuardDuty.

Exam domains: secure architectures 30%, resilient 26%, high-performing 24%, cost-optimised 20%. 65 questions, 130 minutes, pass mark 720/1000.

Key points

- Multi-AZ = availability; read replica = read scale
- NAT gateway lets private subnets reach the internet outbound only
- Set a billing alarm in week 1

Common mistakes

- × Only watching videos; do the labs
- × Leaving NAT gateways running (they cost hourly)

Interview questions

- ☐ Design a highly available web app on AWS.
- ☐ Security group vs NACL?

Learn from

AWS SAA-C03 exam guide (aws.amazon.com) · freeCodeCamp SAA course (YouTube) (youtube.com) · AWS Skill Builder (free tier) (skillbuilder.aws)

Related: AWS IAM and least privilege · Load balancing, replication and sharding · High-level system design (HLD)

L2 · INTERVIEW-READY AI

LLMs and RAG apps ↑ Index

> **WHY IT MATTERS FOR YOU**
> Your AI track from 14 Dec (≤1 h). An AI + security project is a rare combination on a fresher resume.

The concept

An LLM predicts the next token from a prompt; it knows only its training data and what you put in the context window.

RAG (retrieval-augmented generation): split documents into chunks, embed them as vectors, store them in a vector index; at question time, embed the question, retrieve the closest chunks and put them in the prompt so the answer is grounded in your data.

Tool calling lets the model request a function (search, database query) that your code runs. Evals: a fixed set of questions with expected answers to measure changes.

Key points

- Chunk size and overlap matter more than the model
- Always cite retrieved sources in answers
- Evaluate before and after every change

Common mistakes

× No evaluation set
× Putting secrets in prompts

Interview questions

☐ How does RAG work?
☐ How would you evaluate an LLM feature?

Learn from

DeepLearning.AI short courses (free) (deeplearning.ai) · Anthropic docs (docs.anthropic.com)

Related: Prompt injection and AI security

L3 · ADVANCED AI

Prompt injection and AI security ↑ Index

> **WHY IT MATTERS FOR YOU**
> Where your AI and cyber tracks meet. Few freshers can talk about it.

The concept

Prompt injection: untrusted text (a web page, an email, a document in your RAG store) contains instructions that the model follows instead of the developer's. Direct injection comes from the user; indirect injection hides in retrieved content.

Risks grow with tools: an injected instruction can make an agent leak data or take actions. Defences: treat model output as untrusted, least-privilege tools, human confirmation for risky actions, separate trusted instructions from data, output filtering, and red-team tests.

The OWASP Top 10 for LLM applications lists prompt injection, sensitive information disclosure, supply chain, data poisoning, improper output handling, excessive agency and more.

Key points

- Build a test set of injection attempts for your RAG project
- Log and monitor model actions

Common mistakes

× Believing a system prompt alone stops injection

Interview questions

☐ What is prompt injection and how would you mitigate it?

Learn from

OWASP LLM Top 10 (genai.owasp.org) · Gandalf (Lakera) prompt-injection game (gandalf.lakera.ai)

Related: LLMs and RAG apps · OWASP Top 10

L3 · ADVANCED HFT

Probability for HFT interviews ↑ Index

> **WHY IT MATTERS FOR YOU**
> The later HFT track (Apr–Jul 2027). It also makes maths-heavy OA questions easy.

The concept

Counting: permutations, combinations, stars and bars, inclusion–exclusion. Conditional probability and Bayes' rule: $P(A|B) = P(B|A) \cdot P(A) / P(B)$.

Expectation is linear even for dependent variables: $E[X + Y] = E[X] + E[Y]$. Indicator variables turn counting questions into sums of probabilities (expected number of fixed points in a random permutation = 1).

Condition on the first step for recursive questions: expected flips to get HH satisfies a small system of equations (answer 6). Gambler's ruin and random walks follow the same idea.

Key points

- Zetamac mental arithmetic: 40+ then 50+
- Fair price of a game = its expected value
- Write every solved puzzle's key idea in one line

Common mistakes

× Assuming independence without checking

Interview questions

☐ Expected number of rolls to see a 6?
☐ You flip until HH; expected flips?
☐ Bayes: a test is 99% accurate for a 1% disease; P(sick | positive)?

Practise

Brainstellar (brainstellar.com) · Zetamac (arithmetic.zetamac.com)

Learn from

Harvard Stat 110 (projects.iq.harvard.edu) · Blitzstein & Hwang (free book) (projects.iq.harvard.edu)

Related: Competitive programming and contests

L1 · CORE CAREER

The interview loop (UMPIRE) ↑ Index

> **WHY IT MATTERS FOR YOU**
> A fixed routine for every DSA interview so nerves don't decide your structure.

The concept

Understand: restate the problem, ask about input size, ranges, duplicates, empty input; agree on 2 examples. Match: name the pattern. Plan: describe the approach in words and give its complexity; get a nod. Implement: clean code, good names. Review: dry-run an example and an edge case. Evaluate: complexity, trade-offs, what you'd improve.

Always say the brute force first. It shows you can solve it, gives you a fallback and often reveals the optimisation.

Key points

- Spend ~5 min understanding, ~5 planning, ~20 coding, ~5 testing in a 45-min round
- If stuck, think aloud about what you know; interviewers give hints to people who talk

Common mistakes

× Coding before the interviewer agrees with the approach

Learn from

CodePath UMPIRE guide (guides.codepath.org)

Related: Speaking while you code · Patterns, not problems · Mock interviews

L1 · CORE CAREER

Online assessments (OAs) ↑ Index

> **WHY IT MATTERS FOR YOU**
> The first filter for almost every off-campus role. Many strong candidates fail here from poor tactics, not poor skill.

The concept

Typical format: 2–3 DSA problems in 60–90 minutes on HackerRank, HackerEarth, CodeSignal or Mettl, sometimes with MCQs on CS fundamentals and aptitude.

Tactics: read all problems in the first 2 minutes; solve in order of confidence; submit a brute force for partial test cases if the optimal idea isn't coming; check overflow (long long), empty and max-size inputs; stop optimising in the last 10 minutes.

Practise in the same environment: do timed sets on HackerRank, not only LeetCode, because I/O handling differs (you often write main() and read input yourself).

Key points

- Keep your C++ template ready to paste
- Proctoring: webcam, no tab switching
- Simulate a full 90-minute OA weekly from June

Common mistakes

× Spending 60 minutes on problem 1
× Forgetting to read input in the given format

Learn from

HackerRank interview prep kit (hackerrank.com)

Related: Competitive programming and contests · The error log · Time and space complexity

L2 · INTERVIEW-READY CAREER

Mock interviews ↑ Index

> **WHY IT MATTERS FOR YOU**
> Interviewing is a separate skill from solving. Mocks are where you find out what breaks under pressure.

The concept

From 31 May, two mocks a week: one peer DSA mock and one Core CS / design mock. Rotate roles: being the interviewer teaches you what they look for.

Free options: Pramp (peer matching), friends from college or Discord groups, the AI mock-interviewer prompt. Record yourself when practising alone.

After each mock, write three things: what went well, what broke, one fix for next time.

Key points

- Score yourself 1–5 on problem solving, code, communication
- Do mocks at the same time of day as real interviews

Common mistakes

× Only doing mocks with friends who go easy on you

Learn from

Pramp (pramp.com) · interviewing.io (free practice when available) (interviewing.io)

Related: The interview loop (UMPIRE) · Speaking while you code · Behavioural interviews and STAR stories

L1 · CORE CAREER

Behavioural interviews and STAR stories ↑ Index

> **WHY IT MATTERS FOR YOU**
> Amazon's bar raiser and every HR round use behavioural questions. Six good stories cover almost all of them.

The concept

STAR: Situation (one line of context), Task (your responsibility), Action (what you did — most of the answer, in 'I' not 'we'), Result (outcome with a number, plus what you learned).

Prepare six stories that each fit several questions: a hard technical problem (TenantWall IAM), a failure, a conflict, learning something fast (C++/AWS), going beyond the task (a security finding), helping or leading someone.

For Amazon, map each story to Leadership Principles: Ownership, Dive Deep, Learn and Be Curious, Bias for Action, Deliver Results, Customer Obsession.

Key points

- Keep each story to ~2 minutes
- Have a 60-second 'tell me about yourself'
- Real numbers beat adjectives

Common mistakes

× Talking about the team instead of your actions
× No result or lesson

Interview questions

☐ Tell me about a time you failed.
☐ Tell me about a conflict with a teammate.
☐ Why this company?

Learn from

Amazon Leadership Principles (amazon.jobs)

Related: Mock interviews · TenantWall (your research project)

L1 · CORE CAREER

The one-page resume ↑ Index

> **WHY IT MATTERS FOR YOU**
> Off-campus, the resume is read for 6–10 seconds. It decides whether any of your work gets seen.

The concept

One page, single column, no photos, tables or graphics (ATS parsers break on them), exported as PDF named Firstname_Lastname_SDE.pdf.

Order for you: contact links (LinkedIn, GitHub, LeetCode/Codeforces) → education (CGPA if ≥ 7.5) → experience → projects (TenantWall first) → achievements (ratings, CTF ranks, AWS SAA, paper) → skills.

Bullet formula: action verb + what you built + technology + measurable result. 'Built X using Y, which achieved Z.'

Keep a 2-page private master resume with every bullet; cut 3 one-page versions (SDE, Security/AppSec, Cloud) and tailor each application in 5 minutes.

Key points

- Numbers in every project bullet
- Link a live demo or video
- Have two seniors review it

Common mistakes

× Listing every technology you've touched
× Tutorial projects (to-do app, weather app)

Learn from

Overleaf resume templates (Jake's resume) (overleaf.com)

Related: Projects that get shortlisted · Referrals and LinkedIn outreach

L2 · INTERVIEW-READY CAREER

Projects that get shortlisted ↑ Index

> **WHY IT MATTERS FOR YOU**
> Two strong projects with numbers beat six tutorials, and give you the stories for interviews.

The concept

Every project needs: a live link or demo video, a README with an architecture diagram, at least one number (latency, requests/s, users, bugs found), and a 'what I'd do next' line.

For your profile: TenantWall (cloud security research), a rate-limited URL shortener on AWS with load-test numbers (backend + design), a log-based intrusion detector (security + data), a secure RAG assistant with prompt-injection tests (AI + AppSec), and 3+ merged open-source PRs.

Threat-model each project and fix one issue; it becomes a unique interview story.

Key points

- Pin them on GitHub with clean READMEs
- Write tests for core logic
- Never commit secrets (.env in .gitignore)

Common mistakes

× Unfinished projects with no README

Interview questions

☐ Walk me through your project's architecture.
☐ What was the hardest bug?
☐ How would it scale?

Learn from

roadmap.sh project ideas (roadmap.sh)

Related: TenantWall (your research project) · The one-page resume · Threat modelling (STRIDE)

L1 · CORE CAREER

Referrals and LinkedIn outreach ↑ Index

> **WHY IT MATTERS FOR YOU**
> The strongest off-campus channel: the Eternal (Zomato) story in this guide came through a referral.

The concept

Find employees or alumni at the target company on LinkedIn (search 'company + RAIT' first, then any engineer). Send a short, specific note with the job ID and your resume link. Ask for a referral, not 'a job'.

Make it easy to say yes: one-line intro, two proofs (problems solved, project), the exact role and job ID, a resume link. Thank them either way and follow up once after 5–7 days.

Build the network before you need it: comment thoughtfully on engineers' posts, share your project write-ups.

Key points

- 10 referral messages a week
- Alumni reply far more often than strangers
- Track every message in your sheet

Example

```
Hi [Name], I'm a 3rd-year CSE (Cybersecurity) student; I've solved 450+ DSA problems and built [project in one line].
I'm applying for [Role, Job ID] at [Company]. Would you be open to referring me?
Resume: [link]. Thank you!
```

Common mistakes

- × 'Hi, any openings?' messages with no context
- × Messaging the same person repeatedly

Related: The application pipeline · The one-page resume · Cold emailing startups

L2 · INTERVIEW-READY CAREER

Cold emailing startups ↑ Index

> **WHY IT MATTERS FOR YOU**
> Startups hire interns without formal drives. A specific email to a founder can skip the whole funnel.

The concept

Target funded Indian startups (Inc42, Entrackr funding news, YC India companies). Email the founder or CTO directly.

Five lines: you use or studied their product; who you are; one relevant thing you built (with a number); one specific observation (a bug, a missing security header, a feature idea); the ask (internship for N months, 15-minute call). Attach or link resume and GitHub.

Send 5 a week, personalised. A security observation done responsibly (no exploitation) is a strong opener for a cyber student.

Key points

- Subject line: role + your hook
- Follow up once after a week

Common mistakes

- × Mass-mailing the same template
- × Testing their systems without permission

Learn from

Inc42 funding news (inc42.com) · Y Combinator companies (India filter) (ycombinator.com)

Related: Referrals and LinkedIn outreach · The application pipeline

L1 · CORE CAREER

The application pipeline ↑ Index

> **WHY IT MATTERS FOR YOU**
> Off-campus hiring is a numbers game. A batched system makes 50 good applications a month take 90 minutes a week.

The concept

Collect: saved job alerts on LinkedIn and Naukri, 40 bookmarked careers pages checked Monday and Thursday, Unstop and HackerEarth for hiring contests.

Apply in two 45-minute batches (Mon, Thu): tailor the resume in 5 minutes, apply within 48 hours of posting, send the referral message for each job.

Track everything in one sheet: company, role, date, channel, contact, status, next action. Follow up once after 5–7 days. Expect a 2–5% reply rate; volume plus quality produces interviews.

Key points

- Apply even before you feel ready (≈150 problems + 1 project)
- Hiring contests skip resume filters
- Re-use 'why this company' answers

Common mistakes

- × Applying only on campus
- × No tracker, so no follow-ups

Learn from

Unstop (unstop.com) · LinkedIn Jobs (linkedin.com)

Related: Referrals and LinkedIn outreach · Cold emailing startups · The one-page resume

L2 · INTERVIEW-READY CAREER

Preparing for a specific company ↑ Index

> **WHY IT MATTERS FOR YOU**
> Once you have an interview, one focused week beats general practice.

The concept

1) Read 5–10 recent interview experiences for that company and role on LeetCode Discuss and GeeksforGeeks; note round types and repeated questions. 2) Solve its most frequent recent LeetCode-tagged problems (top 30). 3) Map its values to your STAR stories. 4) Use the product: find one improvement and one security thought to mention.

Ask the recruiter about the round structure; they usually tell you.

Key points

- LeetCode company tags need Premium; free alternatives are Discuss posts and GitHub lists of company questions
- Glassdoor interview section

Common mistakes

× Only solving tagged problems without fundamentals

Learn from

LeetCode Discuss: interview experiences (leetcode.com) · GfG interview experiences (geeksforgeeks.org)

Related: Behavioural interviews and STAR stories · Mock interviews

L2 · INTERVIEW-READY CAREER

Offers, CTC and negotiation ↑ Index

> **WHY IT MATTERS FOR YOU**
> Freshers often lose money or sign bad bonds by not knowing what to ask.

The concept

CTC (cost to company) includes fixed salary, variable/bonus, joining bonus, stock (RSUs vesting over years), benefits. In-hand monthly pay is fixed salary minus tax and PF, divided by 12. Ask for the breakdown in writing.

Check the service agreement or bond clause, notice period, location and relocation support before signing.

Negotiation: research ranges on levels.fyi and Glassdoor; avoid naming a number first; if you have a competing written offer, mention it politely. Mass-recruiter fresher offers are rarely negotiable; product companies sometimes adjust joining bonus.

Key points

- RSUs: 4-year vesting is common, often with a 1-year cliff
- PPO = pre-placement offer from an internship

Common mistakes

× Comparing CTCs without separating fixed and variable

Interview questions

☐ What are your salary expectations?

Learn from

levels.fyi (levels.fyi) · Glassdoor India (glassdoor.co.in)

Related: The application pipeline

L3 · ADVANCED CAREER

Open source and GSoC ↑ Index

> **WHY IT MATTERS FOR YOU**
> A brand that removes the college question (see the tier-3 MCA student's GSoC story), and real code review experience.

The concept

Pick one organisation whose tech you use (C++, Python, security tools). Set up the project locally, read the contribution guide, start with 'good first issue' labels, and join their chat.

Aim for 3+ merged PRs before applying to programmes. GSoC contributor applications usually open in March–April (check the timeline each year); LFX Mentorship runs several cohorts a year; MLH Fellowship is another route.

Your proposal should show you already understand the codebase: link your merged PRs and give a week-by-week plan.

Key points

- Quality of PRs > quantity
- Maintainers often refer good contributors for jobs

Common mistakes

- × Typo-fix PRs to many repos
- × Starting a month before the deadline

Learn from

Google Summer of Code (summerofcode.withgoogle.com) · LFX Mentorship (mentorship.lfx.linuxfoundation.org) · MLH Fellowship (fellowship.mlh.io)

Related: Projects that get shortlisted